Institut Polytechnique
Des Sciences Avancées

81 Av. de Grande Bretagne
31300 Toulouse

Laboratoire de Mécanique
des Fluides de Lille

Av. Paul Langevin
59650 Villeneuve-d'Ascq

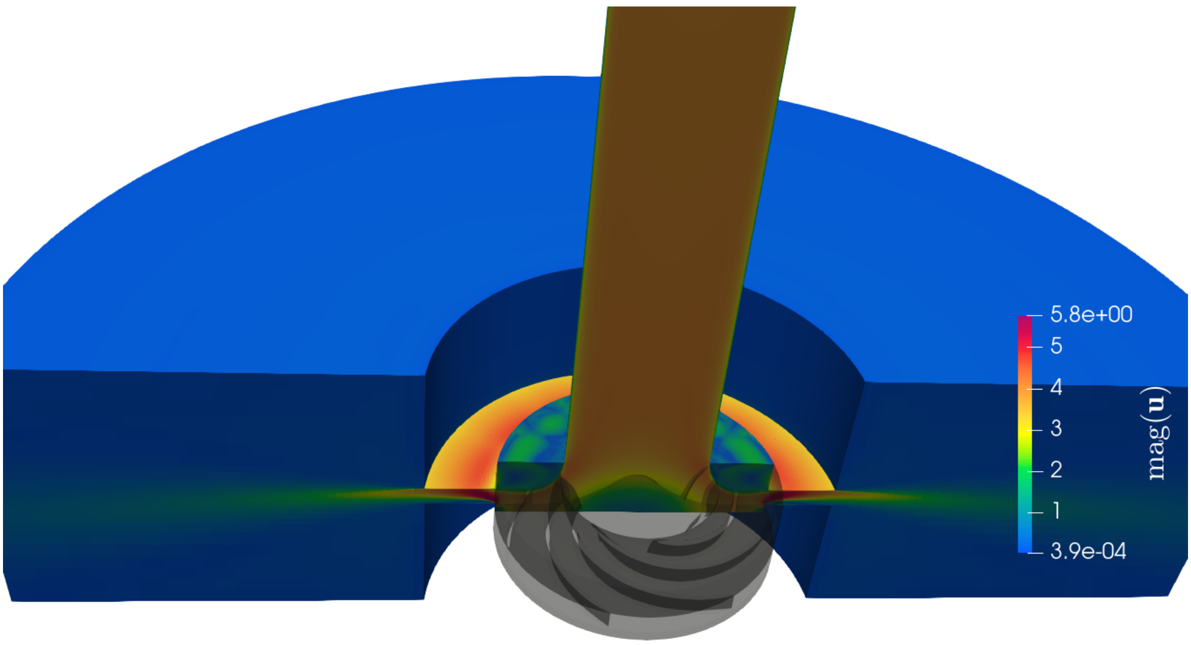
École Nationale Supérieure
d'Arts et Métiers

8 Bd Louis XIV
59000 Lille

End-of-study thesis for engineering degree

Optimization of the performance of a centrifugal pump using Data Assimilation

Intern in Rotating Machines



Cross-section of the pump model

Educational tutor: Samy LABSIR
Laboratory tutors: Marcello MELDI, Antoine DAZIN, Miguel MARTINEZ VALERO

Clément THIBAULT - Aéro 5 From February 26th, 2025 to September 2nd, 2025

Version of August 20, 2025

Acknowledgements

Acknowledgements

First and foremost, I would like to sincerely thank my three internship supervisors. I am deeply grateful to Miguel Valero Martinez for his continuous support, patience, and guidance throughout this internship. His expertise and encouragement have been truly invaluable and will continue to inspire me throughout my thesis and beyond. Many thanks as well to Marcello Meldi for welcoming me into the LMFL and for his insightful advice at every step. I know I can count on you in times of difficulty. I would also like to warmly thank Antoine Dazin, whose clear physical explanations helped me grasp the key concepts behind the experimental pump.

I am especially grateful to Tom, my office mate, for his generosity and clarity in explaining complex topics. His help greatly deepened my understanding on many subjects. Thanks to Paolo for his valuable support and for introducing me to CONES. To Sarp, thank you for your energy and positivity, which made the lab a very enjoyable place to work.

I'd also like to thank my LMFL colleagues Bo, Eddy, Francis, Vincent, Jeanne, Thibaut, and Ernesto for their kindness and camaraderie throughout this experience. Many thanks as well to my friends Dimitri, Benjamin, and Joan for their help with physics, mathematics, and numerical methods.

I am also grateful to the administration teams at ENSAM and LMFL for their support, and to the TGCC for providing access to HPC resources. My thanks to the ANR-JCJC (Agence Nationale de la Recherche — Jeunes Chercheuses et Jeunes Chercheurs), specifically ANR-JCJC-2021-IWP-IBM-DA, which funded this internship.

I warmly thank my family and friends for their constant encouragement throughout my studies and this internship. Their belief in me has been a constant source of motivation.

Last but not least, thank you to Samy LABSIR for agreeing to be my IPSA tutor for this internship and for always being available when needed.

Contents

Summary sheet

The aim of this technical assessment is to summarize the gaps and their causes between the course's missions and what has been achieved. It also enables us to suggest what could be done by the next person working on the same subject.

Summary sheet	Clément THIBAULT - Aéro 5 TIE
Internship topic	**Objectives**
Optimization of the performance of a centrifugal pump using Data Assimilation	Optimize the forcing parameters of an IBM[1] in a centrifugal pump using DA with CONES, to find the correct performance of the pump.
Main customer	**Used tools**
- LMFL - ENSAM	OpenFoam 9, CONES, VSCode, LaTeX, Git, Python,C++, MPI, Bash, TGCC, Paraview
Conducted studies	
- Coding the DA test case in OpenFoam and CONES - Convergence of the CFD solution (physical and numerical stability/precision) - Convergence of the EnKF, adding tools as inflation	
Results	**Explanation of possible differences**
- Drop up to 50% of the mathematical NRMSE with DA (almost 100% expected) - Implementation of inflation, in CONES - Implementation of radial expansion, "azimuthal interpolation" in OpenFoam 9 - Discovering of a recirculation bubble in the diffuser	- After the diffuser, the impeller's wake direction is reversed because of the MRF method - The rotating impeller shape combined with the IBM and MRF allows non-physical flow that induce a recirculation bubble. - Results of the EnKF are not as good as expected because of the IBM method combined to MRF method, which is not adapted to the centrifugal pump geometry.
Encountered difficulties	**Work to be continued**
- Understanding the tools OpenFOAM and CONES - Code everything directly on the TGCC terminal - Initialize physical field of complex instationary simulation	- Use the same type of DA for the analysis of axial compressor internal flows - Synchronize PIV and simulation in the DA - Implement interpolation IBM method

Note: Acronymes are define later.

[1]All acronyms are defined from the next page or available in the glossary.

General Introduction

Since I was young, I've always been fascinated by fluid mechanics. I remember watching planes flying and being amazed by the way they could fly. I was also interested in the way water flows in rivers and sailing boats. In high school, I began working on plane-related projects and explored them more theoretically through Computation Fluid Dynamics (CFD).

This fascination led me to study physics and mathematics at university, where I discovered applied mathematics including CFD, Artificial Intelligence (AI) and Kalman Filter. During my studies, I've become passionated about applied mathematics and fluid mechanics. That's why I did my last internship at the "Centre Européen de Recherche et de Formation Avancée en Calcul Scientifique" (CERFACS). It convinced me to do a thesis in CFD after "Institut Polytechnique des Sciences Avancées" (IPSA), my current engeneering school.

The subject proposed by the "Laboratoire de Mécanique des Fluides de Lille" (LMFL) really suited me: CFD using an open-source library, coupled with AI and experimental data. So it was with enthusiasm that I started this internship on February 26, 2025, in this laboratory for a duration of six months.

Briefly, the LMFL is a public laboratory, consisting of 38 researchers and around 25 PhD students. The laboratory is present on three sites located in Lille ("École Nationale Nupérieure d'Arts et Métiers" (ENSAM) and "Office National d'Études et de Recherche Aérospatiales" (ONERA)) and on the Villeneuve d'Ascq campus (Centrale Lille, "Centre National de la Recherche Scientifique" (CNRS) and University of Lille).

My main tasks during this internship were:

- Ensuring the convergence of the CFD solution;
- implementing the Data Assimilation (DA) problem in Open source Field Operation And Manipulation (OpenFOAM) and Coupling OpenFoam with Numerical EnvironmentS (CONES);
- optimizing the forcing parameters of the Immersed Boundary Methods (IBM) using DA with CONES to correct a non-physical behavior.

To present the details of my internship work, I will first introduce the LMFL research laboratory. In the second part, I will describe the work I carried out, structured into the following six sections:

1. State of the art;
2. Numerical ingredients;
3. Experimental data and pre-processing;
4. IBM preliminary results;
5. Application of the DA;
6. Results.

Chapter 1

The LMFL laboratory and Arts et Métiers Lille campus

1.1 Scientific environment and host institution

The LMFL is a public research laboratory jointly supervised by Centrale Lille, Université de Lille, Arts et Métiers ParisTech, ONERA, and the CNRS.

Figure 1.1: LMFL facilities; Top:Villeneuve d'Ascq campus, bringing together Centrale Lille, the CNRS, and Université de Lille. Bottom-left: ONERA. Bottom-right: Arts et Métiers campus (ENSAM) (source : lmfl.cnrs.fr)

Over the past six months, I have been hosted by ENSAM, specifically at the Lille campus. Arts et Métiers is a prestigious engineering school focused on mechanical, industrial, and energy engineering. The Lille campus gives to students a dynamic and innovative environment in a region with a strong industrial heritage.

The LMFL was created on January 1, 2018, from the merger of two research units: the "Écoulements Tournants et Turbulents" team from the Lille Mechanics Laboratory, and the "Expérimentation et Limite de Vol" unit. This diversity of expertise enables the research teams to engage in numerous collaborative projects with regional, national and international partners.

Research conducted within the LMFL is organized into three main topics:

- **Turbulence**: Studying and modeling turbulent flows using numerical and experimental methods. The focus is on inhomogeneity and non-stationarity in wall-bounded flows, wakes, jets, and various other flow configurations. This research area includes ten permanent researchers, as well as an equal number of PhD students and postdoctoral researchers.
- **Rotating flows**: Experimental and numerical studies of rotating machines (turbomachines, rotors, propellers) operating under degraded conditions, as well as flow control, cavitation, and multiphase flows, are also key topics. This subject gathers six researchers and about ten PhD and postdoctoral students.
- **Flight dynamics in unsteady and non-uniform environments**: Development of analytical and experimental methods to determine the evolving behaviour of low-speed aircraft in perturbed flight domains. Post-stall dynamics, environmental interactions, and experimental design are among the main research interests. This topic involves about five research engineers and ten PhD and post-doctoral students.

About the LMFL we can also find an interesting documents from "Haut Conseil de l'Évaluation de la Recherche et de l'Enseignement Supérieur" (HCERES). This organization is in charge of evaluating all research structures. Every five years, it publishes a report on the LMFL. The last one[1] was published in February of this year. It presents the laboratory's strengths and weaknesses, as well as its research activities, collaborations, workforce, and future perspectives. It highlights the laboratory's commitment to excellence in research, its multidisciplinary approach, and its contributions to the field of fluid mechanics and related areas. More precisely, the report emphasizes the laboratory's strengths in experimental and numerical methods, its expertise in turbulence and rotating flows, and its contributions to the understanding of complex fluid phenomena.

Some key takeaways from the report:

- The facilities are unique, with wind tunnels and turbomachinery test rigs that allow advanced experimental investigations of external and internal flows.
- It is one of the most active European laboratories in the field of next-generation aircraft design, particularly on electric propulsion systems, supported by high-level experimental setups such as the Eiffel wind tunnel (L1) and a vertical wind tunnel with a rotating balance. The LMFL also leads European research on airship aerodynamics and urban drone flight.
- Its experimental platforms are completed by advanced numerical simulations, with equipment partly acquired through regional "Contrat de Plan État-Région" (CPER) programs. The lab conducts original research on turbulence dissipation mechanisms in non-homogeneous, unsteady flows and on turbulent/non-turbulent interfaces.
- Efforts have been made to create synergy between the lab's three geographically separated sites through transversal research themes in optical metrology, flow control, and data analysis. Weekly LMFL webinars support internal collaboration.
- Despite some staff departures, LMFL has maintained its attractiveness by recruiting top researchers, strengthening historical themes like turbulence and instabilities, and expanding into areas like two-phase aerodynamics and data-driven modeling.
- LMFL's scientific output is of high quality, with publications in top journals (e.g., Journal of Fluid Mechanics (JFM)) and participation in major conferences. While flight dynamics

[1] https://www.hceres.fr/sites/default/files/media/publications/rapports_evaluations/pdf/E2026-EV-0590349J-DER-ER-DER-PUR260025134-ST5-LMFL-RF.pdf

publications are fewer, this area is expected to grow through collaborations, particularly with the turbulence team.

- Finally, the lab brings in solid external funding from ERC, ANR, Clean Sky, etc., and works closely with industry partners like CNES and Ariane Group. The leadership was praised for keeping the lab well-organized and for encouraging collaboration across research themes.

1.2 Social and environmental responsibility

ENSAM's Corporate Social Responsibility (CSR), or "Responsabilité Sociétale des Entreprises" (RSE), action plan is structured around five strategic axes[2]:

1. **Strategy, governance and territorial anchoring**: presents ENSAM's actions in terms of sustainable development, in particular the Evolutive Learning Factories 4.0 project aimed at modernizing training and research platforms by fully integrating the CSR dimension;
2. **Education and training**: Training students in eco-responsibility;
3. **Research and innovation**: Ensure that all research themes have direct or indirect impacts on the achievement of international and national sustainable development goals. Ensure free access to research results and data;
4. **Environmental management**: ENSAM aims to reduce its environmental impact by improving energy efficiency, cutting emissions (including Scope 3), and integrating carbon accounting. It also promotes biodiversity and sustainable mobility through staff engagement, responsible purchasing, and green-space management;
5. **Social policy**: Reconciling economic, societal, and environmental dimensions by training staff, teachers, research professors, and stakeholders in sustainable development issues, developing a quality of life policy within the institution, and ensuring equal opportunities in training, research, and human resources. An indicator is the "Qualité de Vie au Travail" (QVT), that is detailed in the "Rapport Social Unique" (RSU)[3] 2024.

The "Comité Social et Économique" (CSE) is the staff representative body within ENSAM. Although specific information on the CSE is limited to the 2022 RSU mentioning the monitoring of the actions of the "Comité d'Hygiène, de Sécurité et des Conditions de Travail National" (CHSCTN) which is the old name of the CSE.

Arts et Métiers also publishes its activity reports, its social balance sheet / single social report and public data then transmitted to the "Commission des Titres d'Ingénieur" (CTI) available online [4]. More specifically, students from Arts et Métiers created a webpage to learn about general CSR and sustainable development: [5] (addressing themes such as ecology, inclusion, gender equality and the fight against discrimination,...).

[2]https://artsetmetiers.fr/fr/ecole-engagee, https://dp-www.s3.ensam.eu/public/2024-11/24_032_plaquette-bilan-RSE_V9_0.pdf
[3]https://dp-www.s3.ensam.eu/public/2025-07/RSU%202024_VCA.pdf
[4]https://artsetmetiers.fr/fr/observatoire-des-donnees
[5]https://artsetmetiers.fr/fr/actualites/developpement-durable-et-rse-faites-le-plein-de-connaissances-sur-savoir

My personal experience of the work environment has been very positive. The working atmosphere within the laboratory is pleasant and conducive to research. The researchers are passionate about their work and share their knowledge with enthusiasm. The team is close-knit and it collaborates with researchers from all over the world. The working atmosphere suits me: researchers generally work hard, but there is no pressure regarding working hours. The building is a beautiful old, typical Lille building, but well renovated and adapted to the research work being conducted (both numerical and experimental). The laboratory offers spacious areas, interior courtyards, large lecture halls, and a large workshop for experiments.

Chapter 2

Optimizing the performance of a centrifugal pump with data assimilation

To introduce the topic of my internship, let's take a simple example: the weather forecast. Meteorologists use data assimilation to improve weather predictions by combining numerical models with real-time observations. This process helps correct model errors and provides more accurate forecasts.

To understand fluid behaviour, scientists rely on physical measurements and governing equations. While analytical solutions exist for simple cases, more complex flows require numerical simulations. However, high-fidelity simulations remain too computationally expensive for industrial use. This limitation has led to the development of approximate models involving tunable parameters. These parameters are calibrated for specific flow conditions, using experimental data. Techniques such as the Immersed Boundary Methods (IBM), which will be detailed later, use a global approach, while the physics of the flow cans locally be very different. That's why it rely on such high-fidelity data calibration. Manual tuning is feasible for a small number of parameters but becomes inefficient and inaccurate as complexity increases. This justifies the use of mathematical approaches like the EnKF, which motivated the definition of my internship topic.

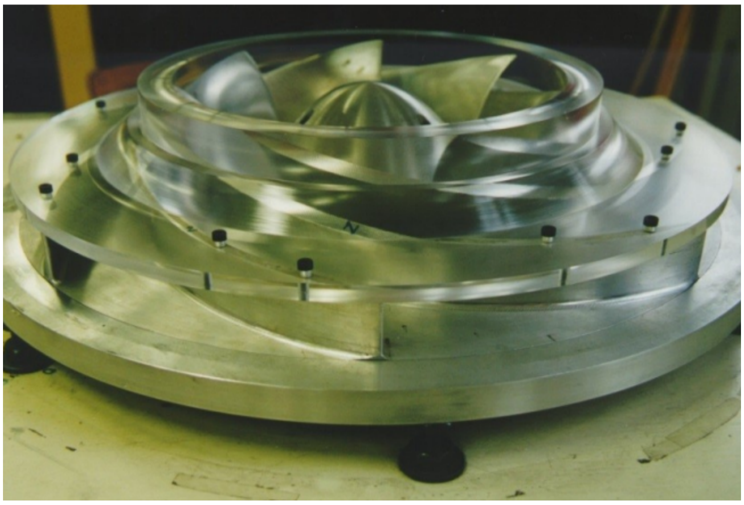
The internship began in February 2025. I was provided with a fix computer, credentials to connect to the "Très Grand Centre de Calcul du CEA" (TGCC) (Irene - Skylake and Rome partitions) and Cassiopée supercomputers, and the link to the Coupling OpenFoam with Numerical EnvironmentS (CONES)'s gitlab[1]. Marcello MELDI, Antoine DAZIN and Miguel MARTINEZ VALERO who are my internship supervisors, showed me around the lab. Then, Tom MOUSSIE and Miguel MARTINEZ VALERO, my co-officies, explained me the basics of Open source Field Operation And Manipulation (OpenFOAM) and CONES, the DA code. I was able to run a few tests on OpenFOAM on the supercomputers, which allowed me to familiarize myself with the tools and the code.

[1] https://gitlab.ensam.eu/pe431/cones-dev

2.1 State of the art

The pump Centrifugal pumps are used since 1475[11] and are still widely used today in various applications, such as water supply, irrigation, turbomachines, and industrial processes. They are known for their low cost, efficiency and reliability, making them a popular choice for many fluid transport applications. The studied centrifugal pump is a "Société Hydrotechnique de France" (SHF) radial impeller, which is a type of centrifugal pump designed for high flow rates and low head applications. The RESEDA benchmark, located in the lab facilities, is used to study complex flow on centrifugal machines as rotor/stator interactions, instabilities, performance degradation, pressure fluctuation, highly unsteady effects,...[12]

(a) Old centrifugal pump used to move hay (personal picture)



(b) SHF impeller used in the RESEDA benchmark

Figure 2.1: Centrifugal pumps

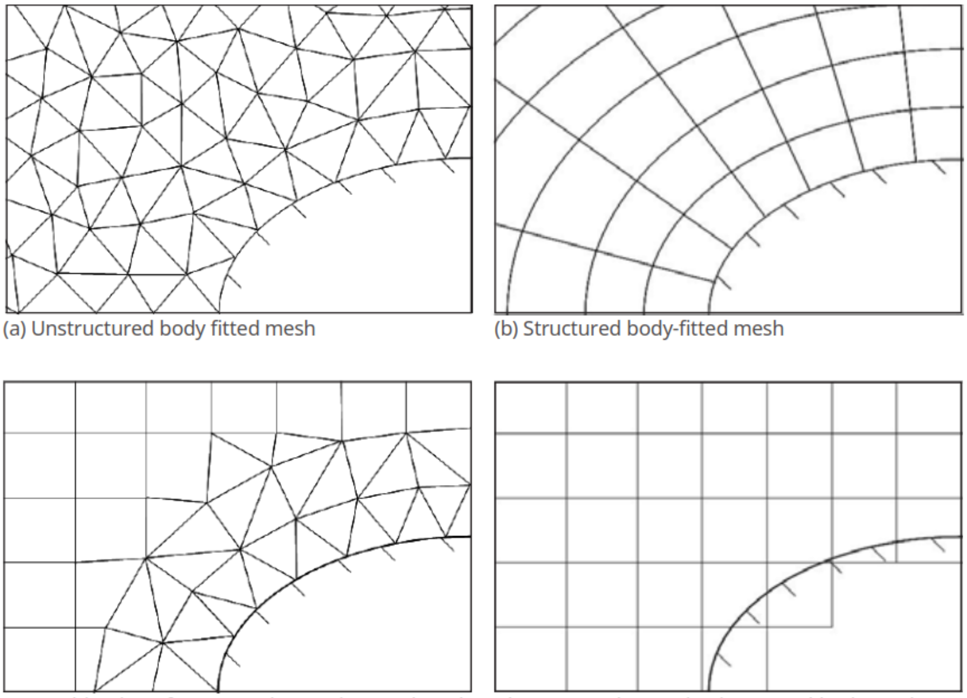
Different high-fidelity data has been recorded on the diffuser of the pump at the LMFL:

- High-Speed Particle Image Velocimetry (PIV) data obtained by Dazin et al. [4] in 2011;
- pressure data obtained by Fan et al. [5] in 2022.

Both experimental (traditionally) and numerical (increasingly in recent years) approchaes have been used to investigate the rotor-stator interactions to improve performance, develop accurate models and permit real-time monitoring and control.

IBM To obtain a numerical simulation of the flow in the pump, a Computation Fluid Dynamics (CFD) can be performed. There are different ways to discretize the space when we do CFD (see Figure 2.2[2]). The most common are structured meshes, unstructured meshes and body-fitted meshes.

[2] https://www.solidworks.com/sw/docs/flow_basis_of_cad_embedded_cfd_whitepaper.pdf



(a) Unstructured body fitted mesh (b) Structured body-fitted mesh (c) Combination of structured Cartesian mesh and non-structured body-fitted mesh near the wall (d) Structured Cartesian immersed-body mesh

Figure 2.2: Differents strategies for meshing

A structured mesh is a mesh whose connectivity between points can be described in a regular manner, typically using multidimensional arrays (1D, 2D, 3D). In such a mesh, the points are organized in a regular grid, allowing easy navigation through the data. In contrast, an unstructured mesh is a mesh where cells cannot be described in a regular manner. They are defined by arbitrary geometric shapes where the connectivity of the points does not follow a regular pattern. These meshes are often used to model complex geometries where a regular grid would be unsuitable.

The body-fitted meshes are the most accurate but also the most expensive in terms of computational resources in case the grids update at each time step.

IBM is a numerical technique that simulates viscous flow with embedded boundaries on grids that do not conform to those boundaries. This allows to keep numerical efficiency on complex geometries and moving boundaries without the need for body-fitted meshes, which can be computationally expensive and difficult to implement.

In 1972 Charles S. Peskin[9] developped the IBM method. His goal was to reduce the computational time, mainly due to the computation of the mesh refinment based on the Courant-Friedrichs-Lewy (CFL) (define Equation 2.19). He added an IBM forcing term on Navier-Stokes equations, only on the simulated artificial heart valve part. He overcomes the numerical instabilitys due to the boundary forces by using an iterative method for solving a fixed-point problem for boundary forces.

In 1999 [2] discovered that using the penalization IMB, the velocity penalized at the wall as the same as the one from a body-fitted approach if **D**, a forcing parameter define in the subsection 2.2.1, tend to infinity.

IBM are usually less accurate and need more cells element to keep good precision in the near-wall region, but more efficient in terms of computational resources when the geometrie is moving. So it's more and more used in CFD [15]. The maths behind the method are detail in the Equation 2.24.

Data Assimilation Data Assimilation (DA) is a technique used to improve the accuracy of numerical models by combining them with observational data[3]. It is widely used in various fields, including meteorology, oceanography, and engineering. The goal of DA is to obtain a more accurate representation of the state of a system by integrating information from different physical measurements or hight-fidelity sources.

There exist many methods for data assimilation, distributed in three main categories: **variational methods**, **statistical methods** and **hybrid** (see Figure 2.3).

- Variational methods (also called classical methods), such as the 3D-Var and 4D-Var, use optimization techniques to find the best estimate of the state of a system by minimizing the difference between model predictions and observations[10];
- Statistical methods (also called sequential methods), such as the Kalman filter and its Ensemble Kalman Filter (EnKF) variant, update the model state iteratively as new observations become available[3]. It is working by maximizing a certain Probability Density Function (PDF) $\langle e^a \rangle_k = 0$;
- And Hybrid methods, combining both variational and statistical that we are not going to explaine here.

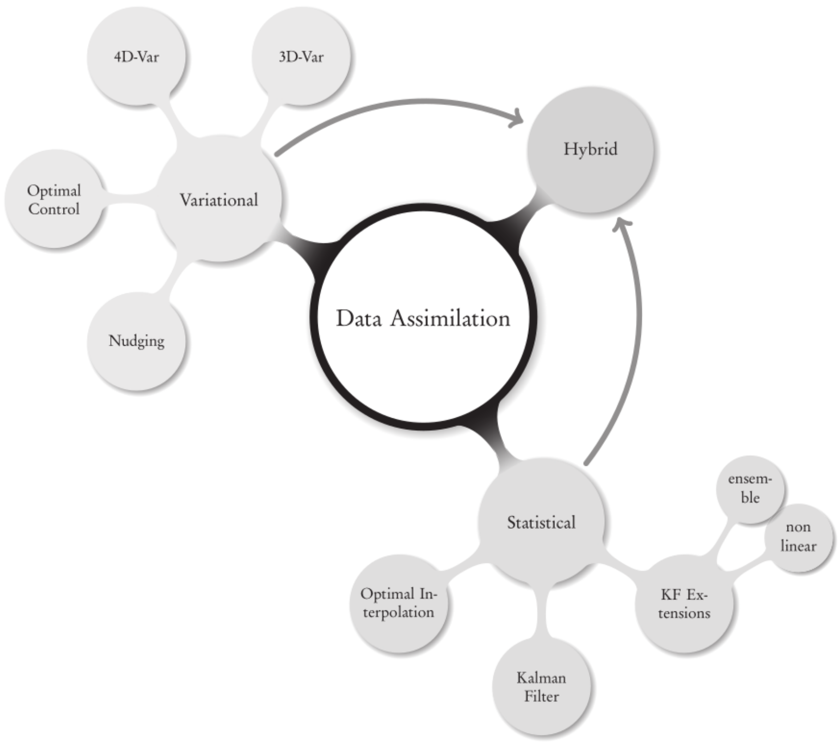


Figure 2.3: The big picture for DA methods and algorithms[3]

EnKF The EnKF, is statistica Monte Carlo DA method that uses a set of model states (reffered to as ensemble member) to estimate the state of the system. The EnKF is particularly well-suited tool for assimilating data on nonlinear and high-dimensional systems and has been successfully applied in various fields, including CFD simulations, allowing for the correction of model predictions based on observed data. It mainly consists of two steps:

- **Forecast**: time advancement of the different model states or ensemble members;
- **analysis**: correction of the model prediction by some measured observation describing the same physical phenomenon.

The maths behind the EnKF method are detailed in the subsection 2.2.3.

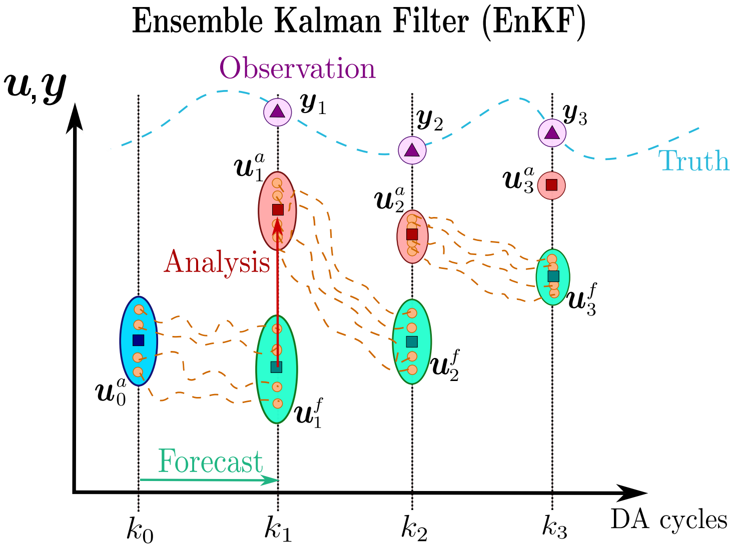


Figure 2.4: Illustration of the EnKF process (100% confidence in observations)

EnKF for DA in CFD has been used at "Laboratoire de Mécanique des Fluides de Lille" (LMFL) for a few years now.

- In 2023, Lucas Villanueva, with Miguel M.Valero and Marcello Meldi from LMFL wrote an article[16] about the amelioration of the flow around a building preforming EnKF on five CFD parameters, showing a global improvement for both the velocity field and the pressure field;
- In 2024, Tom Moussie, with Paolo Errante and Marcello Meldi from LMFL wrote an article[8] successfully optimizes the boudary condition of the academical rectangular 5:1 cylinder benchmarck using EnKF, leading to a significant improvement of the accuracy in the prediction of the turbulent flow;
- This year, Miguel M.Valero and Marcello Meldi published an article combining EnKF, Machine Learning (ML) and IBM on a turbulent channel flow [14]. It shows that this combination can gives very good and low cost results;
- During my internship, Tom Moussie submit an article on the determination of the incidence angle of a 2D wing using EnKF, and Miguel submit also an article about the optimization of forcing parameters of an IBM around an 2D oscillating cylinder, showing very good results.

This led us to explore the use of the EnKF method to optimize the parameters of the IBM forcing term on a centrifugal pump — a more complex case than the previous ones. It combines the 3D turbulent boundary layer characteristics of the turbulent plane channel flow with the moving body configuration of the oscillating cylinder. Its complex geometry requires around 11 million cells. On top of that, rotor/stator interaction in centrifugal machines is highly unsteady and involves turbulent boundary layers and instabilities, making it particularly challenging to model. Traditional CFD approaches often fail to capture these effects accurately. Our goal here is to improve flow prediction at the diffuser by optimizing the IBM forcing parameters, which should enhance the overall simulation and provide a better understanding of the pump's behavior.

In this context, we aim to apply both of these innovative methodologies to enhance the original CFD predictions on a centrifugal pump test case. The approach leverages PIV measurements at the diffuser, along with pressure tap data from the impeller inlet and the diffuser. To achieve this, the manuscript is organized around the following steps:

1. Do a state of the art about the centrifugal pumps, CFD methods including IBM and DA including EnKF, detail in this section 2.1;
2. understand some numerical ingredients as CFD governing equations, the open-source code OpenFoam, the EnKF method, the CONES library and the pump modeling that are explained in the subsection 2.2.1;
3. pre-process the experimental data to be able to compare them to the numerical model, section 2.3;
4. modify OpenFOAM to obtain first simulation results using IBM, section 2.4; ////////////////////////// CHANGER //////////////////////
5. modify CONES to adapte it to the pump case and apply the DA to modify the IBM parameters, section 2.5;
6. analyse the results of the DA on the optimization of the field in the diffuser and the accuracy of the perfomance of the pump, section 2.6.

2.2 Numerical ingredients

2.2.1 CFD governing equations

Navier-Stokes equations CFD is based on the fundamental equations of fluid mechanics. When the fluid is assumed to be Newtonian, these equations reduce to the nonlinear Navier–Stokes partial differential equations.

These equations express three physical conservation principles, given here in their Eulerian form (i.e., fixed spatial reference frame), which is the standard in CFD because it describes how fluid properties evolve at fixed locations in space, which is ideal for mesh-based numerical simulations.

- **Mass conservation (Continuity equation):**

$$\frac{\partial \rho}{\partial t} + div(\rho \mathbf{U}) = 0 \tag{2.1}$$

  with ρ the density and $\mathbf{U}$ the velocity.

- **Momentum conservation (second Newton Law):**

$$\frac{\partial(\rho \mathbf{U})}{\partial t} + \nabla \cdot (\rho \mathbf{U} \otimes \mathbf{U}) = -\nabla P + \nabla \cdot \boldsymbol{\tau} + \mathbf{f} \tag{2.2}$$

  where P is the pressure, $\boldsymbol{\tau}$ is the viscous stress tensor and $\mathbf{f}$ includes body forces (e.g., gravity, source terms).

- **Energy conservation (First law of thermodynamics):**

$$\frac{\partial(\rho E)}{\partial t} + \nabla \cdot (\mathbf{U}(\rho E + P)) = \nabla \cdot (k \nabla T) + \Phi \tag{2.3}$$

  where k is the thermal conductivity, T the temperature, and Φ the viscous dissipation and e the potential energy. $E = e + \frac{1}{2}\boldsymbol{U}^2$ is the total energy per unit mass, with:

  - e: internal energy per unit mass (also called specific internal energy)
  - $\frac{1}{2}\boldsymbol{U}^2$: kinetic energy per unit mass (specific kinetic energy)

  For an ideal gas, the internal energy is generally given by $e = c_v T$, where:

  - c_v is the specific heat capacity at constant volume
  - T is the temperature

Two main indicators of the main behaviour of the fluid are the Mach number (Ma) and the Reynolds number (Re).

The Ma, is defined by the velocity of the fluid divided by the speed of the soud (relative to the fluid). In CFD, fluid is usually considered as incompressible for Mach lower than 0.3. In our case, in atmospheric conditions,

$$Ma = \frac{R_2 \Omega}{c} = 0.095 \quad \textbf{(incompressible)} \tag{2.4}$$

As we consider that we are in **incompressible** flow conditions (because we are at low Ma), Equation 2.1 becomes

$$\nabla \cdot \mathbf{u} = 0 \tag{2.5}$$

And the pressure and velocity are solved iteratively, contrary to compressible flows.

The dimensionless Re measures the ratio between inertial and viscous forces. It indicates if the flow is turbulent, laminar or both, it caratcterizes the flow regime. Let's compute the Re around where the velocity should be the highest.

$$Re = \frac{UL}{\nu} \tag{2.6}$$

U is the velocity of the fluid, L is the characteristic length and ν the kinematic viscosity of the fluid.

$$Re_{r_{tip}} = \frac{R_2^2 \Omega}{\nu} = 5.53 \times 10^5 \tag{2.7}$$

In turbulent flows, a wide range of spatial and temporal scales coexist. Solving the Navier–Stokes equations without any turbulence model (i.e., via Direct Numerical Simulation (DNS)) would theoretically capture all these scales. However, DNS is computationally unaffordable in most practical cases, as it requires resolving down to the smallest dissipative scales, known as Kolmogorov scales.

These Kolmogorov scales define the smallest spatial structures in the flow where kinetic energy is dissipated by viscosity. As shown schematically in Figure 2.5a[3], turbulence consists of large energy-containing eddies, which transfer their energy through a cascade in the inertial range, down to the smallest dissipative structures.

To reduce computational cost, turbulence modelling is employed. Two main approaches exist:

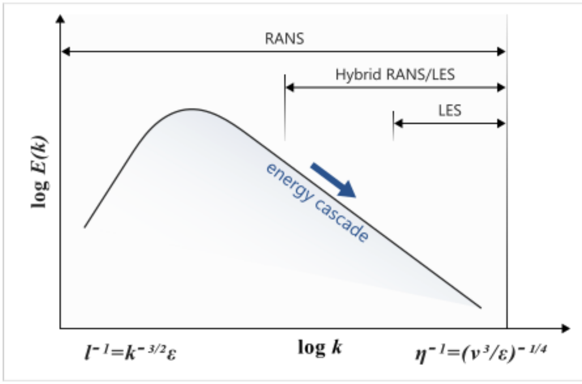
- **Large Eddy Simulation (LES)**: it resolves the large, energy-carrying structures and models only the small-scale turbulence. LES is more accurate than RANS but more computationally expensive, particularly near walls (see Figure 2.5c[4]).
- **Reynolds-Averaged Navier–Stokes (RANS)**: it models the entire turbulence spectrum through statistical averaging and closure models. It is less accurate but significantly cheaper, often used in steady-state simulations.

Note that DNS and LES do not rely on turbulence models in the traditional sense like RANS does; instead, they resolve directly or partially resolve turbulence dynamics. Also, caution must be taken when interpreting visual comparisons of DNS, LES, and RANS fields (see Figure 2.5b)[5], as the underlying assumptions and modelling strategies differ.

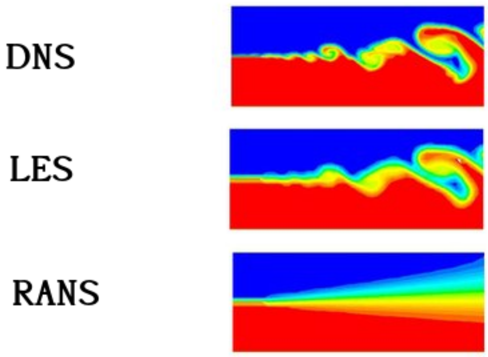
[3] https://help.altair.com/hwcfdsolvers/acusolve/topics/acusolve/training_manual/turbulence_modeling_r.htm

[4] https://www.cfdsupport.com/openfoam-training-by-cfd-support/node346/

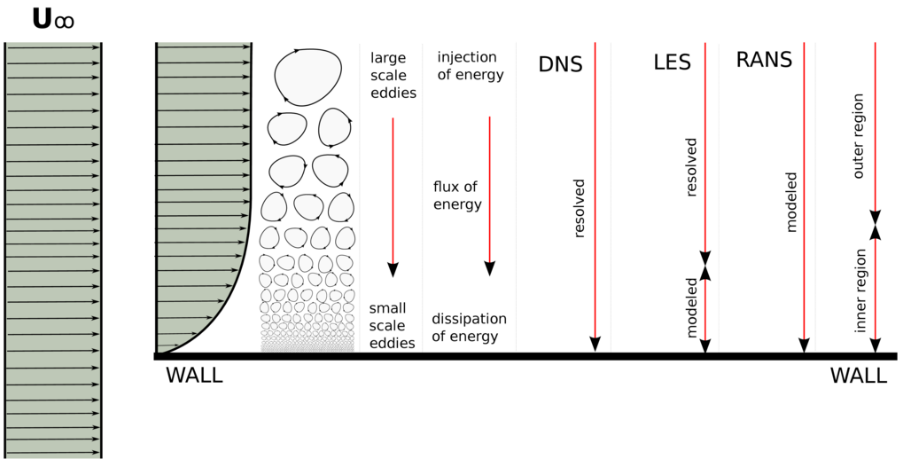
[5] https://blog.diphyx.com/comprehensive-guide-review-to-choosing-the-right-cfd-software-in-2023-features-performance-and-7ebc0623bfa6



(a) Turbulence model on the energy cascade



(b) Illustration of different CFD models



(c) Near wall modelisation of turbulence

Figure 2.5: Turbulence models

In our study, we are going to have a look on the Unsteady Reynolds-Averaged Navier–Stokes (U-RANS) method use the same principle as the RANS method but it's unsteady, allowing to make turn physically the pump in our case.

More precisely, in our RANS and U-RANS simulation, we are going to use a $k-\omega$ Shear Stress Transport (SST) turbulence model. It is a two-equation eddy-viscosity model: it combines the $K-\epsilon$ and the $K-\omega$ turbulence approximations plus a blending region between both.

$K-\omega$ is known to be a good model in the near-wall region but a bad prediction in bulk flow. $K-\epsilon$ makes a good numerical prediction in the bulk flow field but is less accurate near solid surfaces.

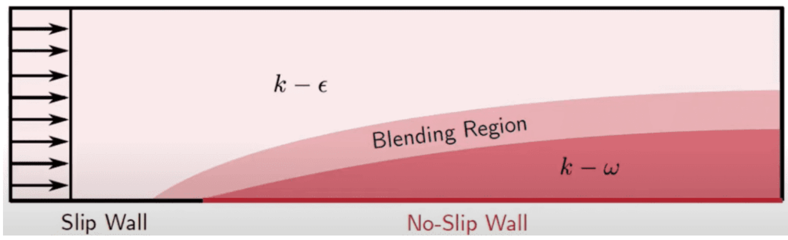


Figure 2.6: $k-\omega SST$ model

$k-\omega SST$ model is defined as follows[6][6][7]:

[6]https://www.cfd-online.com/Wiki/SST_k-omega_model

- Norm of the strain tensor

$$S = \sqrt{2S_{ij}S_{ij}}, \quad \text{avec } S_{ij} = \frac{1}{2}\left(\frac{\partial U_i}{\partial x_j} + \frac{\partial U_j}{\partial x_i}\right) \tag{2.8}$$

- Kinematic Eddy Viscosity:

$$\nu_T = \frac{a_1 k}{\max(a_1\omega, SF_2)} \tag{2.9}$$

- Turbulent kinetic energy production term

$$P_k = \tau_{ij}\frac{\partial U_i}{\partial x_j} \tag{2.10}$$

- Turbulence Kinetic Energy:

$$\frac{\partial k}{\partial t} + U_j\frac{\partial k}{\partial x_j} = P_k - \beta^* k\omega + \frac{\partial}{\partial x_j}\left[(\nu + \sigma_k\nu_T)\frac{\partial k}{\partial x_j}\right] \tag{2.11}$$

- Specific Dissipation Rate:

$$\frac{\partial \omega}{\partial t} + U_j\frac{\partial \omega}{\partial x_j} = \alpha S^2 - \beta\omega^2 + \frac{\partial}{\partial x_j}\left[(\nu + \sigma_\omega\nu_T)\frac{\partial \omega}{\partial x_j}\right] + 2(1-F_1)\sigma_{\omega 2}\frac{1}{\omega}\frac{\partial k}{\partial x_i}\frac{\partial \omega}{\partial x_i} \tag{2.12}$$

- Closure Coefficients and Auxiliary Relations:

$$F_2 = \tanh\left[\left[\max\left(\frac{2\sqrt{k}}{\beta^*\omega y}, \frac{500\nu}{y^2\omega}\right)\right]^2\right] \tag{2.13}$$

$$P_k = \min\left(\tau_{ij}\frac{\partial U_i}{\partial x_j}, 10\beta^* k\omega\right) \tag{2.14}$$

F_1 controls the blending between the $k-\epsilon$ and $k-\omega$ models. F_1 corresponds to the k_ϵ model and $F_1 = 1$ corresponds to the k_ω model.

$$F_1 = \tanh\left\{\left\{\min\left[\max\left(\frac{\sqrt{k}}{\beta^*\omega y}, \frac{500\nu}{y^2\omega}\right), \frac{4\sigma_{\omega 2}k}{CD_{k\omega}y^2}\right]\right\}^4\right\} \tag{2.15}$$

$$CD_{k\omega} = \max\left(2\rho\sigma_{\omega 2}\frac{1}{\omega}\frac{\partial k}{\partial x_i}\frac{\partial \omega}{\partial x_i}, 10^{-10}\right) \tag{2.16}$$

$\phi = \phi_1 F_1 + \phi_2(1-F_1)$

$\alpha_1 = \frac{5}{9}, \quad \alpha_2 = 0.44$

$\beta_1 = \frac{3}{40}, \quad \beta_2 = 0.0828$

$\beta^* = \frac{9}{100}$

$\sigma_{k1} = 0.85, \sigma_{k2} = 1$

$\sigma_{\omega 1} = 0.5, \sigma_{\omega 2} = 0.856$

On the ω region, we can define a near-wall model when the mesh is too coarsed to capture the inner sublayer (see Figure 2.7)[7].

[7] https://2021.help.altair.com/2021/hwsolvers/acusolve/topics/acusolve/training_manual/near_wall_modeling_r.htm

We can define the nomalized wall ditance as:

$$y^+ = \frac{\rho u_{tau} y}{\mu} \tag{2.17}$$

where y is the distance from the wall and μ the fluid dynamic viscosity.

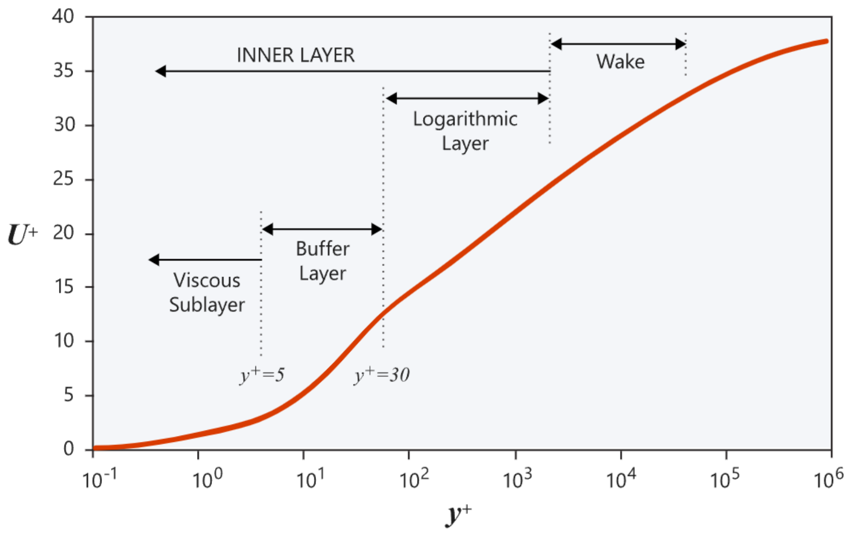


Figure 2.7: Near wall model

MRF method To simulate the rotation of the impeller in a steady-state RANS simulation, we use the Multiple Reference Frame (MRF) method. The MRF strategy consists in solving the Navier-Stokes equations in a rotating reference frame in the rotating region, while keeping the rest of the domain in a stationary (inertial) frame. The mesh and geometry remain static; only the reference frame changes.

The governing equations in the rotating frame (relative velocity) are derived from the Navier-Stokes equations in the inertial frame by applying a transformation that accounts for the rotating frame accelerations.

For an incompressible flow, the Navier-Stokes equations in the rotating frame with relative velocity $\vec{u}_R$ are:

$$\begin{cases} \frac{\partial \vec{u}_R}{\partial t} + \nabla \cdot (\vec{u}_R \otimes \vec{u}_R) + 2\vec{\Omega} \times \vec{u}_R + \vec{\Omega} \times (\vec{\Omega} \times \vec{r}) = -\nabla \left(\frac{p}{\rho}\right) + \nu \nabla^2 \vec{u}_R \\ \nabla \cdot \vec{u}_R = 0 \end{cases} \tag{2.18}$$

Where:

- $\vec{u}_R$ is the velocity in the rotating frame;

- $\vec{\Omega}$ is the angular velocity vector;

- $\vec{r}$ is the position vector;

- ρ is the fluid density;

- ν is the kinematic viscosity.

The additional terms $2\vec{\Omega} \times \vec{u}_R$ (Coriolis force) and $\vec{\Omega} \times (\vec{\Omega} \times \vec{r})$ (centrifugal force) appear due to the non-inertial frame transformation.

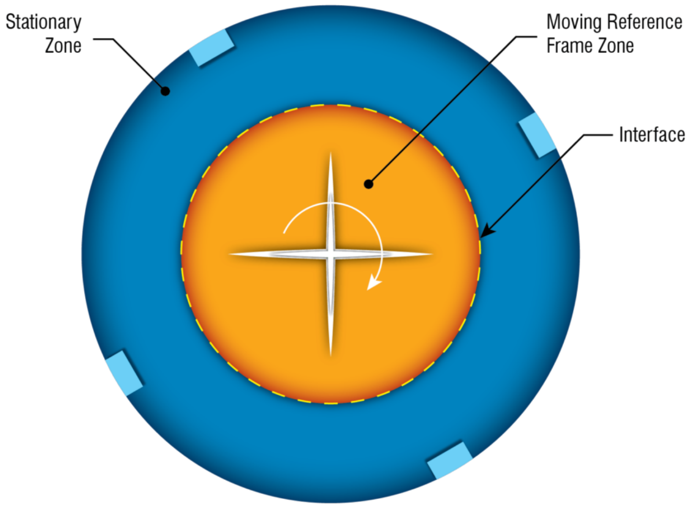


Figure 2.8: MRF strategy: the rotating region is solved in a non-inertial frame

U-RANS In the U-RANS case, the impeller is physically moving.

To do so, the CFL or the courant number has to be computed.

$$CFL = \frac{\Delta_t}{\Delta_x} \times \mathbf{U} \tag{2.19}$$

Typically set to a maximum of 1 for stability.

The CFL number can be interpreted as the fraction of the cell size that a disturbance (or wave) travels during a single time step. In this case, it corresponds to the ratio between the distance covered at the impeller tip in one time step and the spacing between two cells in the azimuthal direction. The CFL is computed locally at each mesh element. Let's compute the Δ_t at the tip of the impeller, where the velocity is the highest and Δ_x the lowest.

- In our model, Δ_x is the distance between two cells in the azimuthal direction at the tip of the impeller, which is given as 1.5 mm or 0.0015 m.

- $\|U_\infty\|$ is the magnitude of the maximal velocity usually "at infinity distance" from the body, which in our case can be approximated by the tangential velocity at the tip of the impeller.

$$\|U_\infty\| = \Omega \cdot r_{tip} \tag{2.20}$$

where:

- ω is the angular velocity in radians per second,

- r_{tip} is the radius at the tip of the impeller.

Assuming the radius at the tip of the impeller is $r = 0.2566$ m, we can compute the tangential velocity:

$$\|U_\infty\| = 125.66 \text{ rps} \cdot 0.2566 \text{ m} = 32.24 \text{ m/s} \tag{2.21}$$

Assuming the distance between two cells in the azimuthal direction at the tip of the impeller is $\Delta_x = 0.0015$ m (which is 1.5 mm), we can now compute the time step Δ_t:

$$\Delta_t = \frac{0.5 \cdot 0.0015 \text{ m}}{32.24 \text{ m/s}} = 2.3 \times 10^{-5} \text{ s} \tag{2.22}$$

IBM equations As seen in the section 2.1, different techniques exist to penalise the immersed part. In our case, we are going to use a classical penalisation method proposed by Angot et al.[2] which enters the family of continuous forcing approach. Instead of using body-fitted mesh, we are going to consider porous medium and add a forcing term on the Navier-Stokes equations (Equation 2.25) to simulate the effect of the immersed boundary on the fluid flow. This term is added only on the penalization part $\Omega_b U \Sigma_b$ (see Figure 2.9).

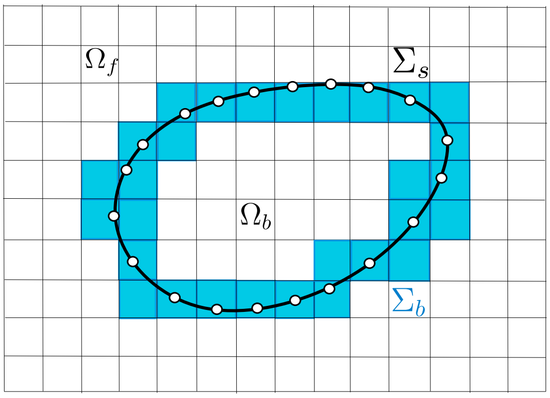


Figure 2.9: IBM regions

The Darcy's law is used in the body interface region Σ_b and the solid region Ω_b. To do so, a spatial dependent tensor $\boldsymbol{D}(\boldsymbol{x})$ must be defined. The resulting penalty term is modelled as follows:

$$\boldsymbol{f}_P = \begin{cases} \boldsymbol{0} & \text{if } \boldsymbol{x} \in \Omega_f \\ -\nu \, \boldsymbol{D} \left(\boldsymbol{u} - \boldsymbol{u_{ib}} \right) & \text{if } \boldsymbol{x} \in (\Sigma_b \cup \Omega_b) \end{cases} \tag{2.23}$$

$\boldsymbol{u_{ib}}(\boldsymbol{x}, t)$ is a target velocity representing the immersed body's physical behaviour. In our case, $\boldsymbol{u_{ib}} = \Omega \times \boldsymbol{R}$. The components D_{ij} of the tensor $\boldsymbol{D}$ are usually controlled to obtain the best compromise between the accuracy and stability of the numerical solver. To converge to the correct result, $\mathbf{D}$ should tend to the infinite. But this lead to numerical instabilities.

We can also write Equation 2.23, once inculding in Naviers-Stokes equations, as:

$$\left(\boldsymbol{u} \cdot \boldsymbol{\nabla} \right) \boldsymbol{u} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \boldsymbol{u} \underbrace{-\xi \nu \boldsymbol{D} \left(\boldsymbol{u} - \boldsymbol{u}_{ib} \right)}_{\boldsymbol{f}_P} \tag{2.24}$$

$$\xi = \begin{cases} 0 & \text{if } \boldsymbol{x} \in \Omega_f \\ 1 & \text{if } \boldsymbol{x} \in (\Sigma_b \cup \Omega_b) \end{cases} \tag{2.25}$$

Here, we are dividing both sides of the equations by ρ for simplification which can be considered uniform as we assume incompressible conditions. So $p = \frac{P}{\rho}$, then $[p] = M \cdot L^{-1} \cdot T^{-2}$ and $[\boldsymbol{u}] = L \cdot T^{-1}$

As we use a RANS turbulence model, in addition to the velocity, we need to penalize the k and ω coefficients.

Finnaly, the forcing terms are defined as follows[8],

$$\boldsymbol{f}_{P_u} = \xi\nu\boldsymbol{D}\left(\boldsymbol{u} - \boldsymbol{u}_{\text{ib}}\right) \tag{2.26}$$

$$\boldsymbol{f}_{P_\mathcal{K}} = \xi\nu D_\eta\left(\mathcal{K} - \mathcal{K}_{\text{ib}}\right) \tag{2.27}$$

$$\boldsymbol{f}_{P_\omega} = \xi\nu D_\omega\left(\omega - \omega_{\text{ib}}\right) \tag{2.28}$$

The problem of this method is that $\mathbf{D}, D_\eta$ and D_ω need to be tuned to obtain the best results. For that, we use the EnKF method to optimize these constants.

2.2.2 The open-source code OpenFOAM

Simulations are performed using the C++ framework OpenFOAM, used to discretize the equations previously presented. OpenFOAM is an finite-volume, open-source code which provides many tools, referred to as executables.

OpenFOAM provides a wide range of tools for solving the Navier-Stokes equations, including solvers, meshing tools, and post-processing tools. It is widely used in the CFD community and has a large user base. It has been developped since 2004 and is today composed of more than a million lines.

- **Pre-processing**, including meshing and decomposing tools;
- **Solving**, including in-built and custom solvers. Each solver is built to answer a specific CFD problem;
- **Post-processing**, including many tools and libraries to compute post-processing operations. OpenFOAM fields can be visualized with `ParaView`, an open-source visualization application.

OpenFOAM is not provided with a graphical interface, which is why a simulation is built by writing "dictionaries", in which is stored the necessary information for the simulation (mesh, turbulence model, maximum number of characteristic times, ...). See tree structure Annex 2.

// Ou si plus lisible: //

[8]https://www.simscale.com/forum/t/defining-turbulent-boundary-conditions/80895, https://www.openfoam.com/documentation/guides/latest/doc/guide-bcs-wall-turbulence-omegaWallFunction.html

Figure 2.10: Tree of the OpenFOAM dictionaries

// Ou si plus lisible: //

```
<case>
├── 0
│   ├── k
│   ├── omega
│   ├── p
│   └── U
├── 0.orig
│   └── 0
├── constant
├── system
│   ├── blockMeshDict
│   ├── controlDict
│   ├── decomposeParDict
│   ├── fvSchemes
│   └── fvSolution
├── tests
└── <time directories>
```

Figure 2.11: Tree of the OpenFOAM dictionaries

The `system` directory contains the main parameters regarding the simulation progress:

– The `controlDict` dictionary defines parameters such as start and end time, time steps, data output formatting, or data average computation;

– the `blockMeshDict` dictionary defines the computational domain, mesh and boundary implementation conditions;

- the `decomposeParDict` dictionary defines the geometry and mesh decomposition, with the objective to run the simulation in parallel, using multiple cores;
- the `fvSchemes` dictionary defines the discretisation schemes used in the Central Processing Unit (CPU)s equations;
- the `fvSolution` dictionary defines the equation solvers, tolerances and other algorithm controls.

The `constant` directory contains the different physical properties of the simulation. Moreover, the subfolder `polyMesh` contains the mesh data, obtained after the `blockMesh` command line is run:

- The `momentumTransport` dictionary defines the simulation type (DNS, RANS or LES) and, in the case of RANS or LES, specifies the corresponding turbulence closure or subgrid-scale model.
- the `transportProperties` dictionary defines the transport model (e.g., Newtonian) and some transport coefficients, such as the kinematic viscosity ν;
- the `MRFProperties` dictionary defined in our case to specify the rotating zone with the origin, rotation axis and rotation speed.

The `0` directory contains the initial solution's fields and the Boundary Conditions (BC) as well. A compiled solver can be called in the terminal of the case directory to run the simulation. If it's called, the `postProcessing` directory stores data from the running simulation, such as forces and probes' data. The `processor0...N-1` directories directly result from the geometry decomposition, which is proceeded through the `decomposeParDict` dictionary. The `dummy.foam` is an empty file which allows to visualize the simulation with `ParaView`.

More precisely, for example, to implemente the IBM forcage term on the speed "u", we can manually modify the momentum equation of the corresponding solver as follows:

```
// Momentum predictor step

MRF.correctBoundaryVelocity(U) // Corrects U at boundaries to include MRF effects
                               // (e.g., rotating walls)

tmp<fvVectorMatrix> tUEqn
(
    fvm::div(phi, U)             // Convective term
  + MRF.DDt(U)                   // Time derivative in the rotating reference frame
  + turbulence->ddivDevSigma(U) // Turbulent diffusion term
  + cmptMultiply(Dprime, U - UIB)    // Forcing term: Dprime * (U - UIB)
 ==
    fvModels.source(U)           // Source terms from external models
                                 // (e.g., body forces), here equal to 0
);
fvVectorMatrix& UEqn = tUEqn.ref();

UEqn.relax() // Applies under-relaxation for numerical stability

fvConstraints.constrain(UEqn); // Applies physical constraints
                               // (e.g., boundary conditions)
```

With

- *cmptMultiply* the element-wise matrix multiplication;
- $Dprime = \xi \nu D$, with $Dprime$ denoting the reduced form of D. The prime symbol is used to distinguish the non-dimensional quantity from its physical counterpart;

- U the flow velocity;
- UIB is the imposed velocity at the walls.

2.2.3 EnKF

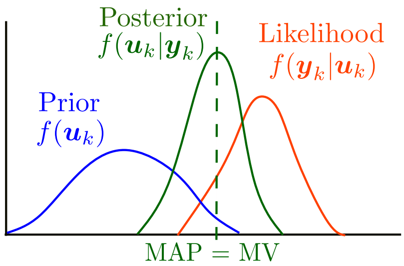


Figure 2.12: Kalman Filter algorithm scheme

As seen in the section 2.1, we are going to use the stochastic EnKF to assimilate the high-fidelity data. But first, let's explaine the Kalman Filter[9].

Kalman Filter The estimation is calculated by using a set of observations $\mathbf{y}$, a prior state $\mathbf{x}$, and a linear model $\mathbf{M}$. The estimation state is obtained through two operations, the forecast and the analysis, respectively using the superscript f and a.

The system state $\mathbf{x}_k$ is as follows:

$$\begin{bmatrix} \mathbf{u}_k \\ \boldsymbol{\theta}_k \end{bmatrix} \in \mathbb{R}^{(n_{\text{vars}} \cdot n_{\text{cells}} + n_\theta) \times m} \tag{2.29}$$

where:

- $\mathbf{u}_k$ is the state matrix (can contain velocities, pressure, temperature...) of the domain at the time instant k, which we don't care about in our case;
- $\boldsymbol{\theta}_k$ are the model coefficients, assumed time-invariant in our case;
- m is the number of ensemble members.

The observation data $\mathbf{y}_k$ is defined as:

$$[\mathbf{y}_k]_{n_{\text{obs}},m} \sim \mathcal{N}(y_k, \mathbf{R}_k) \tag{2.30}$$

Observation data may be built using:

- Velocity, pressure, temperature or other physical observed values;
- Some discret model parameters such instantaneous global force coefficients C_D, C_L, C_f. In our case, it will be five IBM forcing parameters.

Prediction step:

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{M}_k \hat{\mathbf{x}}_{k-1|k-1} + \mathbf{B}_k \mathbf{u}_k \tag{2.31}$$
$$\mathbf{P}_{k|k-1} = \mathbf{M}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^\top + \mathbf{Q}_k \tag{2.32}$$

[9]https://cones-dev-pe431-2dbff22738a4d54c2b947e6fa9a9a77fadd187b3bd1a85c.gitlab-pages.ensam.eu/data_assimilation.html

Update step:

$$\tilde{\mathbf{y}}_k = \mathbf{z}_k - \mathbf{H}_k \hat{\mathbf{x}}_{k|k-1} \tag{2.33}$$
$$\mathbf{S}_k = \mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^\top + \mathbf{R}_k \tag{2.34}$$
$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^\top \mathbf{S}_k^{-1} \tag{2.35}$$
$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k \tilde{\mathbf{y}}_k \tag{2.36}$$
$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1} \tag{2.37}$$

Where:

- $\hat{\mathbf{x}}_{k|k-1}$: predicted state at k
- $\hat{\mathbf{x}}_{k-1|k-1}$: updated state at $k-1$
- $\mathbf{M}_k$: state transition matrix
- $\mathbf{B}_k$: control input matrix
- $\mathbf{u}_k$: control input
- $\mathbf{P}_{k|k-1}$: predicted covariance
- $\mathbf{P}_{k-1|k-1}$: updated covariance at $k-1$
- $\mathbf{Q}_k$: process noise
- $\mathbf{z}_k$: measurement
- $\mathbf{H}_k$: observation matrix
- $\tilde{\mathbf{y}}_k$: innovation
- $\mathbf{S}_k$: innovation covariance
- $\mathbf{K}_k$: Kalman gain
- $\mathbf{R}_k$: measurement noise
- $\mathbf{I}$: identity matrix
- $\hat{\mathbf{x}}_{k|k}$: updated state
- $\mathbf{P}_{k|k}$: updated covariance

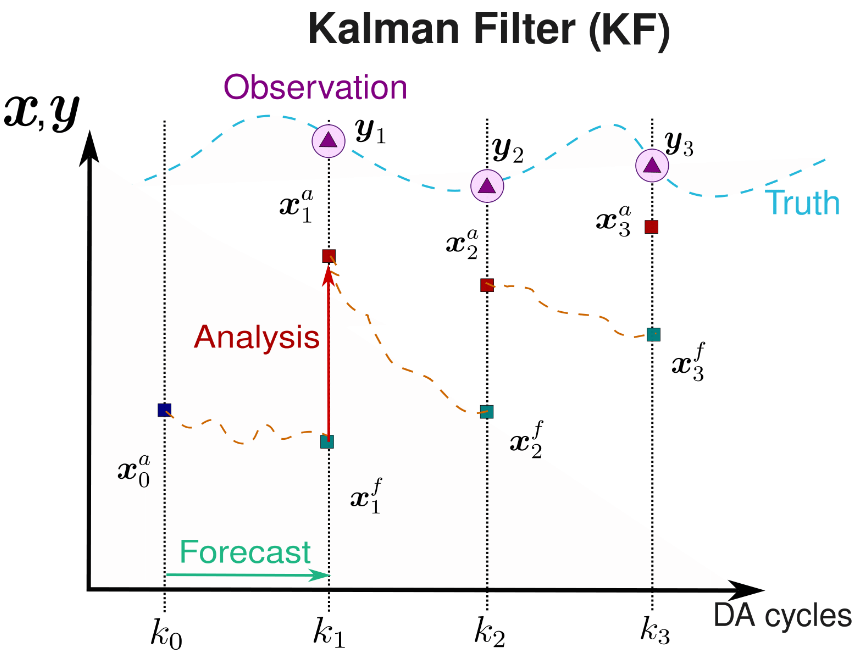


Figure 2.13: Kalman Filter algorithm scheme

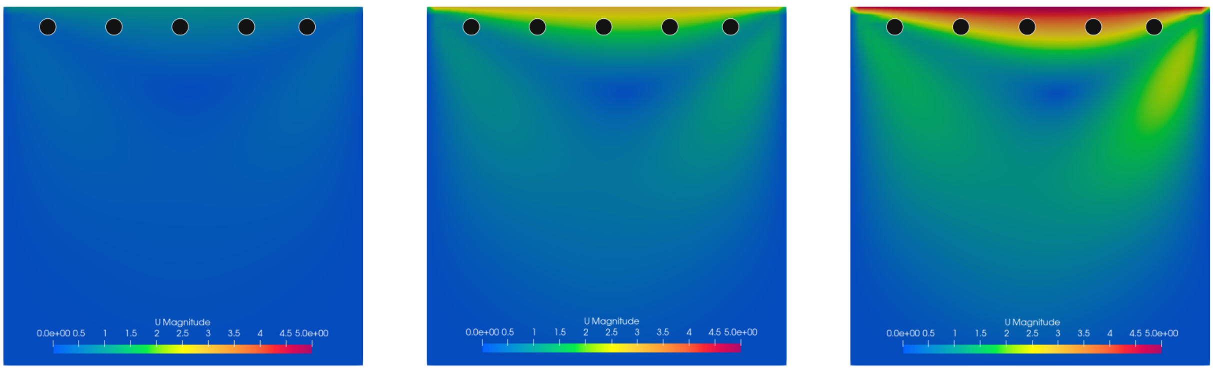
Kalman filter presents two main drawbacks that concern our case. First, as the Extended Kalman Filter (EKF), it only works for moderate deviations from linearity and Gaussianity. If M represents the Navier-Stokes equations, then equations are very not linear. Secondly, for a CFD mesh, the covariance matrix $\mathbf{P}_k$ would have a dimention of $(n_{vars} \times n_{cells}, n_{vars} \times n_{cells})$.

The EnKF allows high dimensional systems by avoiding the explicit definition of $\mathbf{P}_k$ and its Monte-Carlo realistations are much more design for non-linear models. A problem can be the underestimation of $\mathbf{P}_k^a$ but the inflation, define later on this subsection, and localisation, helps mitigate this issue.

Ensemble Kalman Filter We saw previously in this subsection that today, we can theorically run a Direct Numerical Simulation (DNS) which resolves exactly the Navier-Stokes equations, but it is still too expensive to be used in practice. Therefore, we generally use Large Eddy Simulation (LES) or Reynolds-Averaged Navier–Stokes equations (RANS) models to simulate the flow. The EnKF method can be applied to these models to adjust specific model parameters or flow fields, enhancing the overall physical consistency of the simulation.

A good example based on the simple and academic *cavity test case* illustrates it well. The cavity test case is a simple flow in a square cavity, where the flow is driven by a moving lid. The flow is characterized by a recirculation zone and a vortex in the center of the cavity. The EnKF method can be used to assimilate data from the flow, such as velocity or pressure measurements, to improve the accuracy of the simulation. The EnKF method can also be used to estimate the parameters of the model, such as viscosity or turbulence model coefficients, to improve the physical accuracy of the simulation.

In our test case, we are going to optimise the velocity wall condition. To do so, we use the result on 5 locations of a precise converged simulation with a to boundary condition of 5 m.s^{-1} as a reference. We will then assimilate data from the flow, at the 5 observation points (see Figure 2.14a). We start from a simulation with a wall condition of 1 m.s^{-1}, which is not the correct value. The EnKF method will then be used to correct the wall condition by assimilating the data from the flow at the 5 observation points. The EnKF method will iteratively update the wall condition until it converges to the correct value, which is 5 m.s^{-1} in this case. After only three EnKF iterations, the wall condition converges to the correct value (with a relative error of 0.05 m.s^{-1}).



(a) Initial state (b) After one EnKF it. (c) Converged after 2 EnKF it.

Figure 2.14: Evolution of the state through EnKF iterations (●: observation point)

The following matrices are needed to build the EnKF algorithm:

1. **Observation covariance matrix $\mathbf{R}_k$**: we assume Gaussian, non-correlated uncertainty for the observations ($\sigma_{i,k}$ is the standard deviation for the variable i at time instant k).

$$[\mathbf{R}_k]_{n_{\text{obs}},n_{\text{obs}}} = \sigma_{i,k}^2 \, [\mathbb{I}]_{n_{\text{obs}},n_{\text{obs}}}$$

2. **Sampling matrix $\mathcal{H}\left(\mathbf{x}_i^f\right)$**: it is the projection of the model into the position of the observations.

$$\left[\mathcal{H}\left(\mathbf{x}_i^f\right)\right]_{n_{\text{obs}},m}$$

3. **Anomaly matrices** for the system's state $\mathbf{x}_i^f$ and sampling $\mathcal{H}\left(\mathbf{x}_i^f\right) = \mathbf{s}_i^f$ matrices. They measure the deviation of each realization with respect to the mean.

$$\mathbf{X}_i = \frac{\mathbf{x}_i - \bar{\mathbf{x}}}{\sqrt{m-1}}, \quad \mathbf{S}_i = \frac{\mathbf{s}_i - \bar{\mathbf{s}}}{\sqrt{m-1}}$$

   with

$$[\mathbf{X}]_{n_{\text{vars}}\times(n_{\text{cells}}+n_\theta),m}, \quad [\mathbf{S}]_{n_{\text{obs}},m}$$

4. **Kalman Gain matrix**:

$$\mathbf{K} = \mathbf{X}\mathbf{S}^\top\left(\mathbf{S}\mathbf{S}^\top + \mathbf{R}\right)^{-1}$$

   with

$$[\mathbf{K}_k]_{n_{\text{vars}}\times(n_{\text{cells}}+n_\theta),n_{\text{obs}}}$$

For RANS representation, there is no physical time advancement of the model, so each instant is represented through a different interation of the solver.

Algorithm 1 Scheme of the EnKF used in the present analyses for parameter optimization.

1: **Input:** $\mathcal{M}$ (model); $\boldsymbol{R}$ (observation covariance); $\boldsymbol{y}_k$ (observations);
$\boldsymbol{u}'_{i,0} = [\boldsymbol{u}_{i,0}, \theta_{i,0}]^T$ (initial prior states with $\boldsymbol{u}_{i,0} \sim \mathcal{N}(\overline{\boldsymbol{u}}_0, \boldsymbol{P}_0)$ and $\theta_{i,0} \sim \mathcal{N}(\overline{\theta}_0, \boldsymbol{P}_0)$)
$k = 1, 2, ..., K$ is the iteration index; $i = 1, 2, ..., N_e$ is the ensemble index
2: **for** $k = 1$ to K **do**
3: **for** $i = 1$ to N_e **do**
4: Advance in time: $\boldsymbol{u}_{i,k}^f = \mathcal{M}(\theta_{i,k-1})\,\boldsymbol{u}_{i,k-1}$
5: Observation: $\boldsymbol{Y}_{i,k} = \boldsymbol{y}_k + \boldsymbol{e}_i$, $\boldsymbol{e}_i \sim \mathcal{N}(0, \boldsymbol{R})$
6: Compute ensemble means: $\overline{\boldsymbol{u}}_k'^f = \frac{1}{N_e}\sum_{i=1}^{N_e}\boldsymbol{u}_{i,k}'^f$, $\overline{\mathcal{H}(\boldsymbol{u}_k^f)} = \frac{1}{N_e}\sum_{i=1}^{N_e}\mathcal{H}(\boldsymbol{u}_{i,k}^f)$
7: Anomaly matrices: $\boldsymbol{X}_k = \frac{\boldsymbol{u}_{i,k}'^f - \overline{\boldsymbol{u}}_k'^f}{\sqrt{N_e - 1}}$, $\boldsymbol{S}_k = \frac{\mathcal{H}(\boldsymbol{u}_{i,k}^f) - \overline{\mathcal{H}(\boldsymbol{u}_k^f)}}{\sqrt{N_e - 1}}$
8: Kalman gain: $\boldsymbol{K}_k = \boldsymbol{L} \odot \boldsymbol{X}_k^f(\boldsymbol{S}_k)^T[\boldsymbol{S}_k(\boldsymbol{S}_k)^T + \boldsymbol{R}]^{-1}$ /// L to change ///
9: Update: $\boldsymbol{u}_{i,k}'^a = \boldsymbol{u}_{i,k}'^f + \boldsymbol{K}_k(\mathbf{Y}_{i,k} - \mathcal{H}(\boldsymbol{u}_{i,k}^f))$
10: (Deterministic) covariance inflation: $\boldsymbol{u}_{i,k}'^a = \overline{\boldsymbol{u}}_k'^a + \lambda_k(\boldsymbol{u}_{i,k}'^a - \overline{\boldsymbol{u}}_k'^a)$
11: **end for**
12: **end for**

Observation data y can be some local sensors (such as PIV data) or global variables (such as performance)

With $\overline{\boldsymbol{u}}_k'^f = [\overline{\boldsymbol{u}}_k^f, \overline{\theta}_{k-1}]^T$ and $\boldsymbol{u}_{i,k}'^f = [\boldsymbol{u}_{i,k}^f, \theta_{i,k-1}]^T$

////// Choisir entre "u" et "x" ///////////

In our case, $\overline{\boldsymbol{u}}_k'^f = [\overline{\theta}_{k-1}]^T$, $\boldsymbol{u}_{i,k}'^f = [\theta_{i,k-1}]^T$ and $\lambda_k = 0$, $\boldsymbol{y}_k = \boldsymbol{y}$.

Some *deterministic inflation* is included to increase the variability of the state predicted by the EnKF:

$$\boldsymbol{u}_{i,k}^a = \overline{\boldsymbol{u}}_k^a + \lambda_k\left(\boldsymbol{u}_{i,k}^a - \overline{\boldsymbol{u}}_k^a\right) \tag{2.38}$$

Here is an illustration of the deterministic inflation, that increase the variability of the member ensemble to avoid a too fast collaps and permit to have more chance to find a lower local (or in a better case, global) minimum solution:

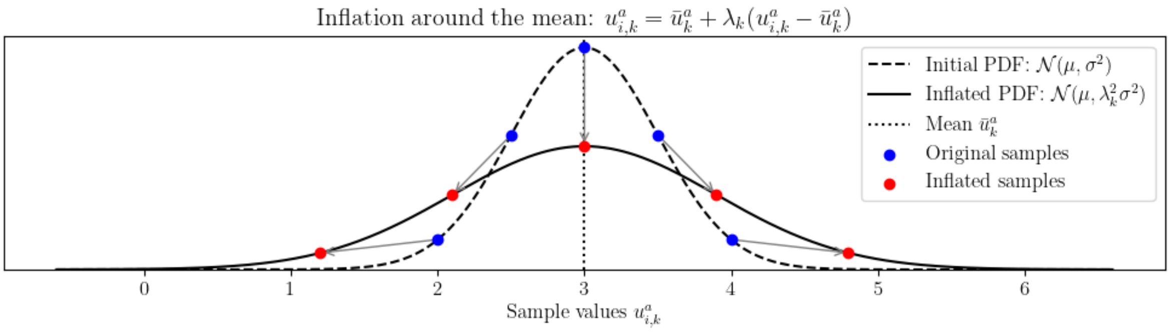


Figure 2.15: Deterministic inflation

I coded it but it finally was not needed for the centrifugal pump case.

2.2.4 The CONES library

CONES is a library coded in Python/C++, that has been developed since 2022 at LMFL. It is the only code that permits online DA in CFD codes (it uses Message Passing Interface (MPI) process to do it). It's parallelized and can be used on supercomputers. It is mainly designed to work with OpenFOAM and DA codes as EnKF.

A specific directory structure must be observed, as outlined in Figure 2.16. Let's take our OpenFOAM/EnKF example as support to explain CONES. Concretly, on one side, m OpenFOAM simulations are running with run at the same time on n processors and on the other side, the EnKF runs on e processors (usually only one is enough). At the begining, all variables are initialized on both sides, using MPI process. Then, all simulation run at same time (with different initial states). Sampled data ($H(u_k^f)$) and $\begin{bmatrix} \mathbf{u}_k \\ \boldsymbol{\theta_k} \end{bmatrix}^f$ are send to the EnKF. Using y_k, EnKF algorithm can compute and send back the updated velocity field (not computed in our case) and the parameters θ_k^a to the different ensemble members (each are different befor converging). With these new parameters, the loop can be done again.

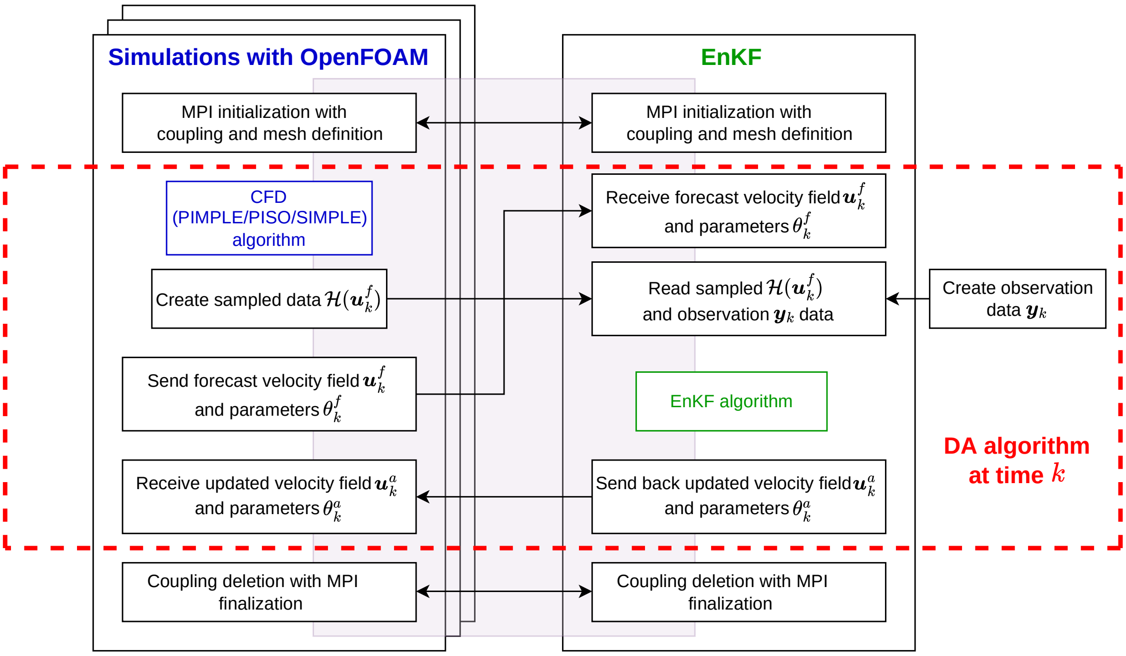


Figure 2.16: CONES structure

2.2.5 Pump modelling

Figure 2.17 illustrates the model of the pump on a yz plane, for $x = 0$. The suction pipe has a length of $H = 10R_1$, allowing the near-wall boundary lawer to develop. Then, a cylinder, where the IBM and MRF will be applied, containing the impeller, is defined (the impeller in .stl can be find on github[10]). At the entrance of the impeller/outlet of the suction pipe junction and at the outlet of the impeller/inlet of the diffuser junction, there was leakage as the whole impeller was moving during the experiment. We assume a zero-leakage as we aren't in a very hight mass flow rate.

In our simulation, the z-axis aligns with the rotational axis, and the reference frame is centred at the impeller. The plane at $z = 0$ divides the diffuser into two identical halves.

[10] https://github.com/fromano88/CentrifugalPump

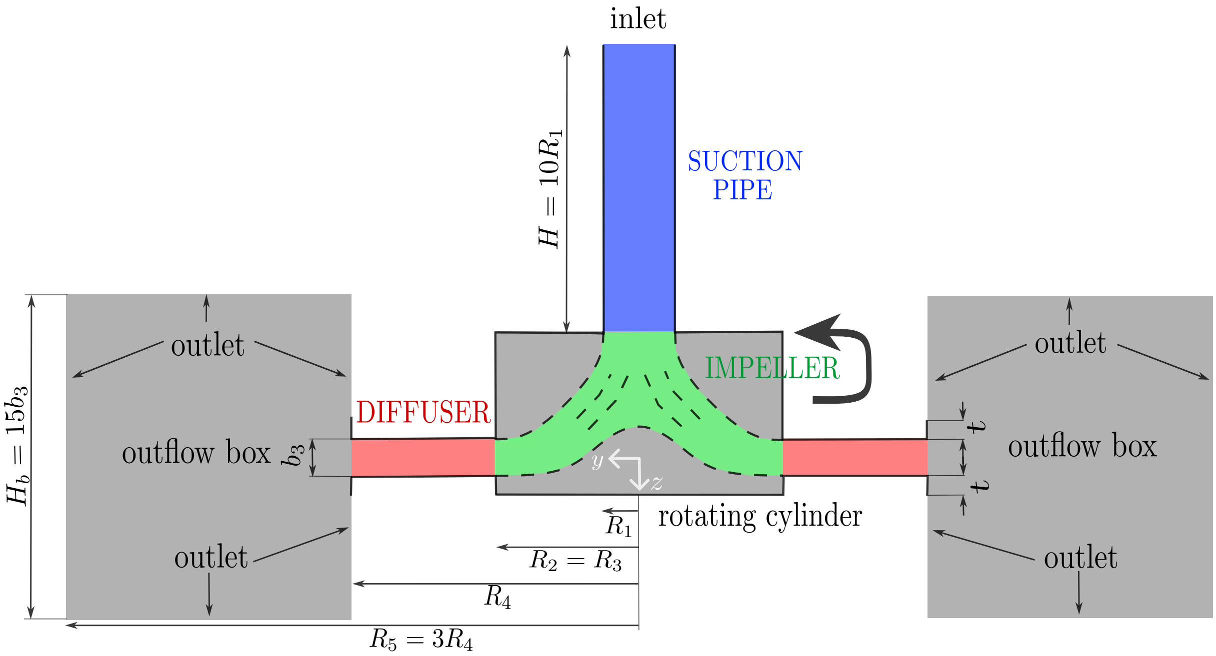


Figure 2.17: Model of the RESEDA benchmark

Boundary conditions are defined as follows:

- Inlet: velocity inlet
- Outlet: pressure outlet
- Wall: no-slip wall

To process CFD, we need to discretize the numerical model of the pump. The mesh is generated using the `blockMesh` utility, which is a built-in OpenFOAM tool. It is composed of around 11 million hexahedral cells. (quasi only, exepted some interface, cf Figure 2.18). We don't need complex mesh because we don't need to fit the geometry of the impeller as we use IBM.

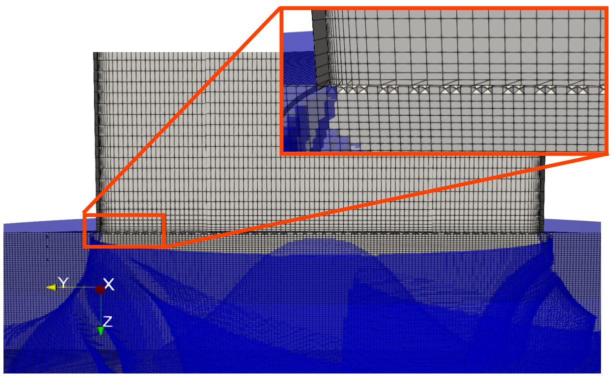


Figure 2.18: Mesh interface between the pipe and the impeller

Regarding the near-wall treatment, at the suction pipe, y + envegale 1, enabling direct resolution of the viscous sublayer and imposition of boundary conditions without wall functions. On the other hand, the highest y + value in the body-fitted regions corresponds to y + envegale 83, so some of the mesh elements (mainly close to the lower wall of the diffuser) are located in the logarithmic region, thus requiring wall functions. Within the rotating cylinder and around the impeller, the mesh resolution yields y + envegale 100. The friction velocity u tau is assumed to be around 20 - 25 times smaller than the peripheral velocity U2 . The blades are discretised to ensure an average of six mesh elements across their solid thickness, preventing excessively thin regions.

The penalisation coefficients used in the Equation 2.26 are defined as follows:

$$\boldsymbol{u}_{\mathrm{ib}} = \cancel{\boldsymbol{u}_r} + \boldsymbol{\Omega} \times \boldsymbol{r} \tag{2.39}$$

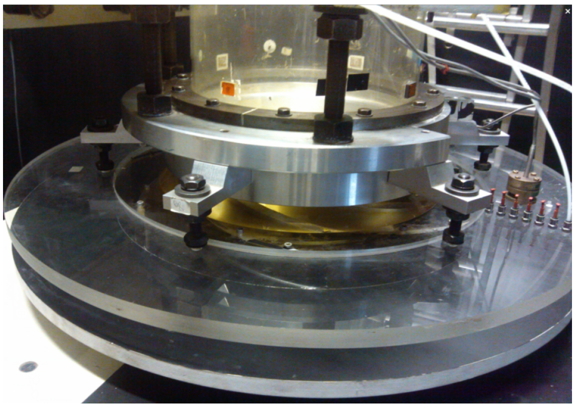
$$\mathcal{K}_{\mathrm{ib}} = 0 \tag{2.40}$$

$$\omega_{\mathrm{ib}} = \left(\omega_{\mathrm{vis}}^2 + \omega_{\mathrm{log}}^2\right)^{1/2} \tag{2.41}$$

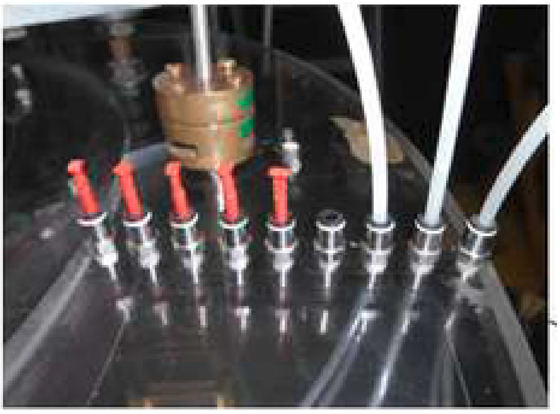
where $\omega_{vis} = \frac{6\nu}{\beta_1 y_{\mathrm{ref}}^2}$ is the viscous sublayer and $\omega_{log} = \frac{\sqrt{\mathcal{K}}}{C_\mu \kappa y_{\mathrm{ref}}}$ is the logarithmic region (see 2.7).

2.3 Experimental data and pre-processing

As discussed in section 2.1, the RESEDA benchmark facility is located at LMFL, where PIV and pressure data have been acquired.



(a) Vaneless diffuser of the centrifugal pump



(b) Pressure taps on the diffuser

Figure 2.19: Diffuser and pressure taps of the centrifugal pump

The operating conditions of the machine were:

	Impeller characteristics	
R_1	Tip inlet radius	$141.1\,mm$
R_2	Outlet radius	$257.5\,mm$
Z	Number of blades	7
Q_d	Design flow rate	$0.236\,m^3/s$
K	Mean blade thickness	$9\,mm$
Ω	Rotating velocity	$1\,200\,rpm$
β_2	Outside blade angle (from peripheral velocity $U = R_2\Omega$)	$22.5°$
	Vaneless diffuser characteristics	
b_3	Diffuser width	$38.5\,mm$
R_3	Inlet radius (without leakage, i.e., $R_3 = R_2$)	$257.5\,mm$
R_4	Diffuser outlet radius	$385.5\,mm$
t	Thickness of the upper and lower sections of the diffuser	$20\,mm$

Table 2.1: Summary of the main geometrical and operating parameters of the impeller and diffuser at design conditions

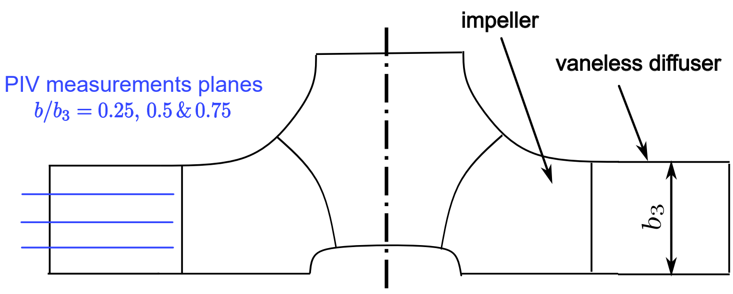


Figure 2.20: Position of the PIV planes on the diffuser

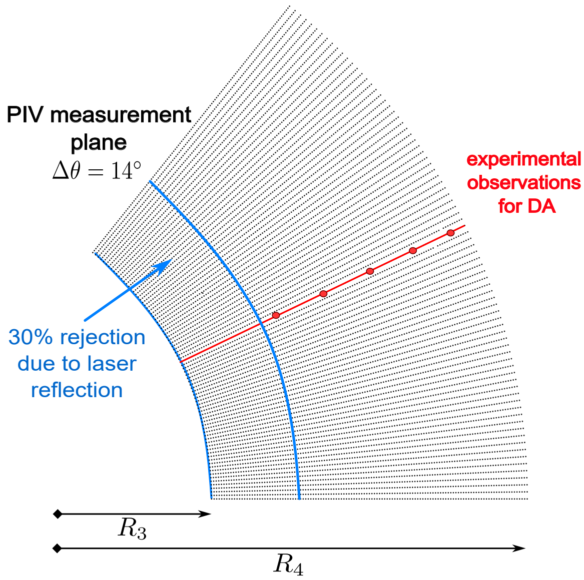


Figure 2.21: Position of the PIV points on a polar slice of the diffuser

PIV data Data is stored in a directory structure that contains six directories. The six directories correspond to two tests realised at 25%, 50% et 75% of the diffuser height. Each test was repeated twice for the same operating condition and the same height (R_1, R_2). In each directory, there are 1581 files corresponding to instantaneous ASCII data file. The ten columns are corrresponding to

$$x(m), y(m), V_x(m.s^{-1}), V_y(m.s^{-1}), r(m), \theta(rad), Vr(m.s^{-1}), V_\theta(m.s^{-1}), V_z(m.s^{-1}), Unknown$$

And the 10,125 lines correspond to the 81 locations over x and 125 locations over y of the PIV. The PIV data is acquired in a rectangular zone of 0.08 m x 0.124 m. The cathesian coordinates are given in a local system, where the absolute origin is at the center of the pump.

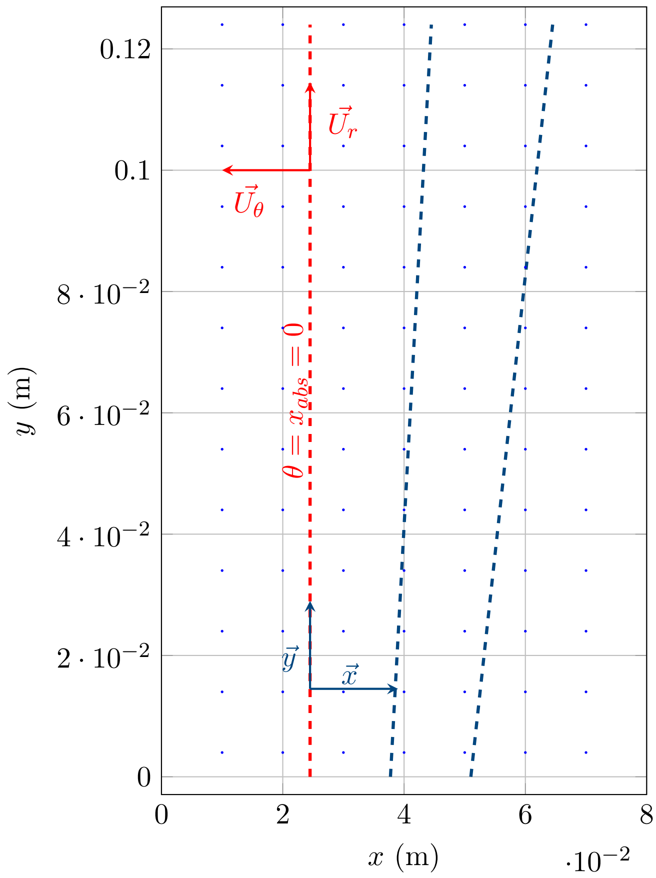


Figure 2.22: PIV acquisition zone in the RESEDA experiment, with orientation of the (r, θ) velocity components.

Pre-processing on the velocity data[11]

In this work, we aim to improve the numerical model of a centrifugal pump using experimental data obtained from PIV. On one side, we have a numerical simulation of the pump (the digital twin), and on the other side, experimental PIV measurements. To combine the strengths of both, we use data assimilation, a method that updates a numerical model using real-world data. We specifically use the EnKF, a data assimilation technique that relies on an ensemble of simulations to estimate and reduce uncertainty in the model. The process can be summarized in two main steps:

1. **Forecast Step** We run a steady-state numerical simulation of the pump using the Multiple Reference Frame (MRF) and the IBM methods. This yields an ensemble of model states, representing the expected flow field within the pump.

2. **Analysis Step** We compare the simulation results with the experimental PIV data. To do this, we interpolate the PIV measurements onto the mesh of the numerical model, creating an ensemble of observation vectors. We then compute the difference between the simulated and measured data, known as the innovation (see mathematical formulation in subsection 2.2.1). This innovation is used to update the ensemble through the EnKF algorithm, improving the numerical model.

The measurements are done as folow:

[11] In the numerical simulation, the z-axis is toward the center of the Earth, unlike in the experiment, where it is upwards. Then, the wheel rotates counterclockwise in the numerical simulation frame and counterclockwise in the experiment. Here, the numerical simulation formalism was the reference for the frame.

1. **Nature of the Numerical Simulation** The simulation is stationary (time-independent) and uses the MRF approach to model rotation. While the velocity field remains constant over time, it varies in space, especially in the diffuser region. Because of the pump's geometry, we expect a periodic velocity pattern that repeats every 1/7th of a revolution (due to the 7 blades). However, this repeating pattern is not clearly visible in unsteady simulations due to wake deformation.

2. **Nature of the Experimental Data** The PIV system captures 49 images per full rotor revolution, meaning there are 7 images between two successive blades. In the rotating frame of reference, each image corresponds to a phase shift of approximately 360°/49.

Ideally, to compare consistent datasets, we could run a time-resolved unsteady simulation and synchronize it with the PIV snapshots. However, for this initial study, we chose to work with a steady MRF simulation. Although we could attempt to reconstruct the spatial velocity field from the phase-averaged PIV data (as done by Dazin et al. [4]), there are two main limitations:

To address these issues, we opted to average the PIV data over time, which is equivalent to spatial (azimuthal) averaging in the rotating frame. On the simulation side, we therefore perform azimuthal averaging of the numerical results to make them consistent with the experimental data. This approach ensures that the data fed into the EnKF, both the forecast and the observations, are comparable and compatible in both spatial and statistical sense.

1. First, we need to determine how to convert the coordinates from the local system to the absolute system. We observe that between lines 55 and 56, the radius remains constant while the angular position θ transitions from below to above π. For the 75% plane, a similar approach is used, but between lines 65 and 66, due to experimental constraints.

We also plotted x versus θ for three different values of θ and found an intersection point at $x = 0.0245$. Therefore, we now know that for $x = 0.0245$, $\theta = \frac{\pi}{2}$.

The absolute coordinate system is then defined by the following equations:

$$\begin{cases} x_{\text{rel}} = x_{\text{abs}} + 0.0245 \\ y_{\text{rel}} = x_{\text{abs}} + R_2 \end{cases} \tag{2.42}$$

with $R_2 = 257.5$ mm

2. Average Since we are dealing with a stationary simulation (time-invariant), we perform a time average of the data. There are approximately 49 velocity maps (i.e., 49 files) per impeller rotation. To compute the time-averaged field, we average over the azimuthal angle θ.These files are distributed across six directories, each corresponding to a different acquisition plane.

3. Sampling We further average the data between radii R_1 and R_2, resulting in three files of size 81×125 lines and 10 columns. At this stage, we retain only the data along the line $x = 0.0245$. More precisely, we average the data between $x = 0.024$ and $x = 0.025$. Finally, the data are truncated at line 89, where the radius exceeds 0.3 m. This truncation is necessary due to laser reflections on the pump casing during the PIV experiments.[12]

[12]Concretely, to average the data over the angle θ, we modify the interpolation function in CONES. Instead of read coords, the interpolation function read the data in a file named `coords_seventh.txt`. Then, it computes the interpolation and average the data over the angle θ (between $-\pi/7$ and $\pi/7$), by doing a somme over θ and dividing by the number of points (150). To go back from 150x5x3x3 (to detail) to 5x3x3 data. All of those computation are mad in a cylindrical referencial.

4.Shaping for CONES CONES keep data in a particular way. We need to convert the data in a format that cones can read. From 3 files (of $10 \times 81 \times 125$ elements) to the files `fieldCylindrical.txt` (of size $n_{\text{obs}} \times n_{\text{dim}} \times n_{\text{vars}}$ lines) and `obs_coordinates.txt` (of size n_{obs} lines and n_{dim} columns).

5.Creation of a file for the interpolation Finally, we need to create the `coords_seventh.txt` file for the interpolation function in CONES. We don't just use the coordinates of the points in the PIV data, beacause we want to average over the angle θ on the pump. The numerical simulation is $2\pi/7$ periodic, so we interpolate in 150 points over theta between 0 and $2\pi/7$, as illustrated in the Figure 2.23. To do that, we just take the `obs_coordinates.txt` file and duplicate 150 times the points over the angle θ (between $-\pi/7$ and $\pi/7$). In our case, we chose 150 that will aproximately correspond to the shap of a cell at this radius. To be consistent with axis further, we take points between $-\pi/7 + \pi/2$ and $\pi/7 + \pi/2$. [13]

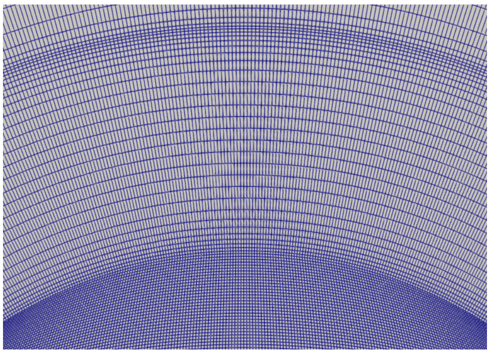


Figure 2.23: Imaginary points to compute the average of the data over the angle θ in the diffuser

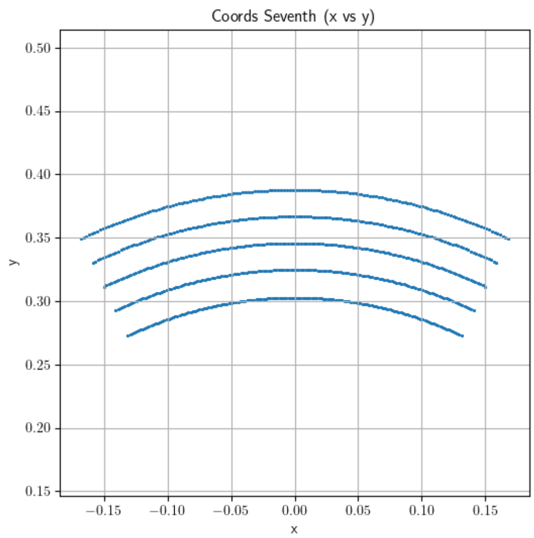


Figure 2.24: Interpolation points over 360/7 degrees

[13] Concretely, to average the data over the angle θ, we modify the interpolation function in CONES. Instead of read coords, the interpolation function read the data in a file named `coords_seventh.txt`. Then, it computes the interpolation and average the data over the angle θ (between -$\pi/7$ and $\pi/7$), by doing a somme over θ and dividing by the number of points (150). To go back from 150x5x3x3 (to detail) to 5x3x3 data. All of those computation are mad in a cylindrical referencial.

Pressure data In addition to the nine pressure tapes on the diffuser, Dazin et al. [4] has also captured (Figure 2.25) the inlet and outlet pressure that allows to compute the global performance of the pump, that match with M. Fan simulation [5]

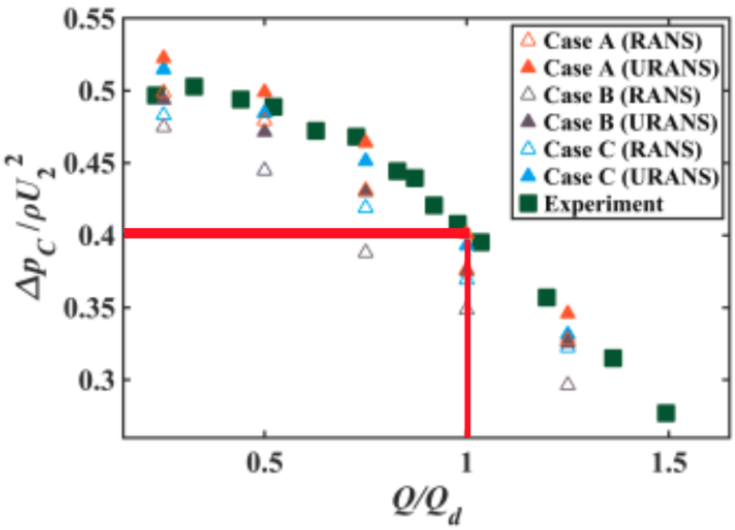


Figure 2.25: Overall pump performance versus reduced pump flow rate Q[4]

As we are at $Q/Q_d = 1$, we can retreive:

$$\frac{\Delta P_P}{\rho U_2^2} = \eta \Leftrightarrow \Delta P_P = \rho U_2^2 \eta \tag{2.43}$$

For the high-fidelity value, we can find experimental data form the same pump [5], that's match with M. Fan simulation (eq to approximately $\eta = 0.4$). So wa can also compute:

U_2 is the impeller tip speed computed from the impeller diameter and the rotation speed of the pump.

$$U_2 = (\Omega \times R2)^2 = (2 \times \pi \times \frac{1200}{60} \times 0.2566)^2 = 32,2m/s \tag{2.44}$$

Finally:

$$\Delta P_p = 1.225 \times 32.24^2 \times 0.4 \approx 415 \text{ Pa} \tag{2.45}$$

2.4 IBM preliminary results

Once the pump model implemented in OpenFOAM, we can run the simulation with the IBM. After empirical tests and with experience of the laboratory, we found that the parameters $\mathbf{D} = (3e7, 3e7, 3e7), D_k = 3e7$ and $D_\omega = 3e7$ are godd starting point for the IBM parameters. And it verify a converged CFD solution.

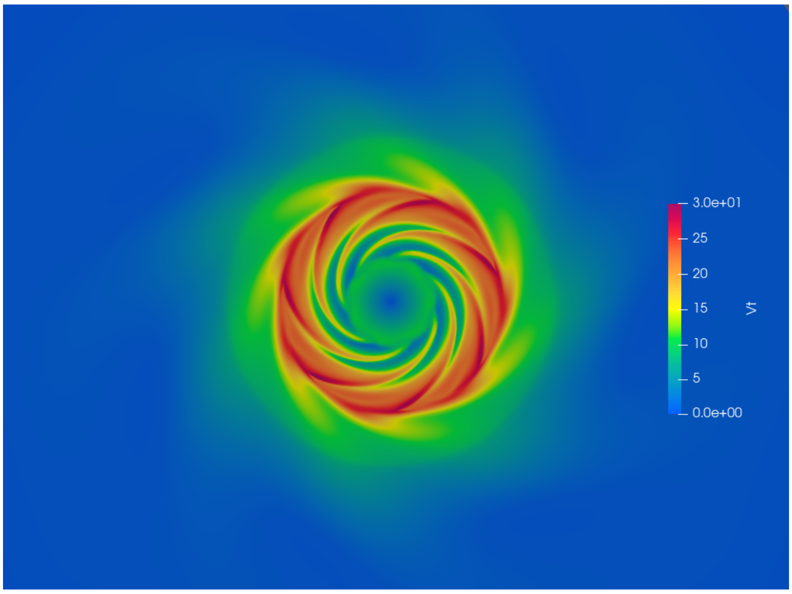


Figure 2.26: First results of the IBM for all parameters equal to 3e7. Slice at $z = 0$ for $\mathbf{D} = (3e7, 3e7, 3e7), D_k = 3e7, D_\omega = 3e7$

With the conditions specified in the Table 2.1, and a value of 3e7 for the all five forcing parameters, we obtain these velocity profils comparing comparing the velocities of the simulation and of the PIV data. Both are averages over time and over the angle θ:

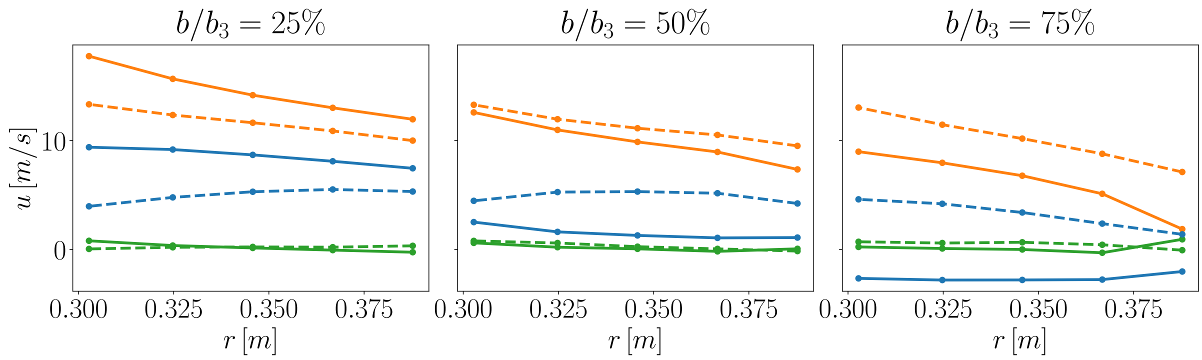


Figure 2.27: Comparison between the profiles inside the diffuser for the numerical simulation (continuous lines), and the PIV data (dashed lines). In particular, (——) represents the radial velocity u_r, (——) depicts the azimuthal velocity u_θ, and (——) illustrates the vertical velocity u_z. $\mathbf{D} = (3e7, 3e7, 3e7), D_k = 3e7, D_\omega = 3e7$

As seen in Figure 2.21, we use five observations points on the three PIV plans. Our goal is to make these curves fit as much as possible.

We already seen that there is a notable difference beteween the experimental and numercial radial velocity in the plan at 75%. The numerical simulation predicts a negative velocity instead of a positive one. This is explained by a non physical recirculation bubble as you can see in the Figure 2.29 deeper exmplained in the folowing part.

A good indicator of the accuracy of the simulation is the performance of the pump. It's directly linked to the pressure difference between the outlet and the inlet. As seen in the previous section 2.3, we should obtain around 415Pa. In our case, we obtain a pressure difference of around 306 Pa, far from what we expect.

2.5 First simple application of the DA

As represented on the Figure 2.16, we need to adapt the OpenFOAM and CONES codes to apply the EnKF on the centrifugal pump case. Among others, the MPI exchanges are coded.

Modifications on OpenFOAM Previously, OpenFOAM has been adapted to our case, including the implementation of the IBM and MRF methods, as well as the definition of the mesh and boundary conditions. Now, we need to adapt OpenFOAM to work with CONES, by including MPI processes taht will allows to run a simulation with variables IBM parameters and that will be able to return interpolated averaged velocities to CONES.

Adaptations on CONES On the other side, although CONES is already adapted to work with OpenFOAM, we need to adapt it to our case. We need to adatp the code to the type of observations.

Test case Finally, we have to initialize a repesitroy for the test case, i.e., the centrifugal pump. This includes defining the initial parameters, the observation points, and the initial state of the ensemble members, including a `conesDict`.

Parameters

To determine the variance for the initial perturbation, two perspectives were considered. The experience at lab shows that an initial perturbation of around 5% is good. From our side, the target was that, on average, an ensemble member differs by approximately 1×10^7 from the previous time step, for parameter values of order 3×10^7. With 30 ensemble members, this corresponds to about 95% of the values (i.e., within 2σ) being contained in the interval $[5 \times 10^7 \pm 1 \times 10^7]$. This leads to a standard deviation of $\sigma = 0.5 \times 10^7$, or about 10%. We finally choose a variance of 5% for the first test. /// A RELIRE //

Let us introduce the observation window. The observation window was defined by observing the number of iterations required for the influence of the IBM forcing to reach the diffuser and take the form of the converged result. Turbulence parameters were not considered for defining this window, since their effect is present throughout the domain and their influence is almost immediate (although their convergence is slower).

The number of iterations between two analysis phases was determined by monitoring how many iterations are required for a parameter perturbation (e.g., 34% change) to significantly affect the solution in the diffuser. What matters here is not full convergence, but that the solution starts to follow the correct trend. For turbulence parameters, this timescale is expected to be much shorter, since the forcing is applied everywhere in the domain. We determined a number of 800 iterations for the observation window.

The `conesDict` and the `controlDict` contains the parameters of the EnKF algorithm, such as the observation window and the following. In our case, we set the number of ensemble members to 30, no inflation, no state optimisation (only parameters), and an absolute observation noise on the velocity of 0.2 m/s (equivalent to 1.5% of the maximum observed velocity). These parameters must be adjusted depending on the specific requirements of the simulation and the characteristics of the data. The number of iterations (800) was determined by testing the convergence trend of a single OpenFOAM simulation when parameters are modified by 34% (i.e., seven times the standard deviation of the initial parameter perturbation). Since the standard deviation may temporarily increase with iterations, care must be taken when choosing this number. An extract of the `conesDict` is given in Annex 3.

First results

After let the simulation run for around 50 iterations, we obtain the first results of the optimisation with CONES. The parameters are optimised to fit the PIV data, and the velocity field is represented on a slice at $z = 0$. The resulting parameters are the folowing: $\mathbf{D} = (4e7, 2e7, 4e7), D_k = 3e7$ and $D_\omega = -3e7$. The sum of the NRMSEs on the velocity field diminished, of around 20%. The global performance of the pump didn't change significantly. Usually, around 200 iterations are needed ton converged but is not need for a first result and cost a lot of CPU time (1270 CPU $\times$ hours per EnKF iteration).

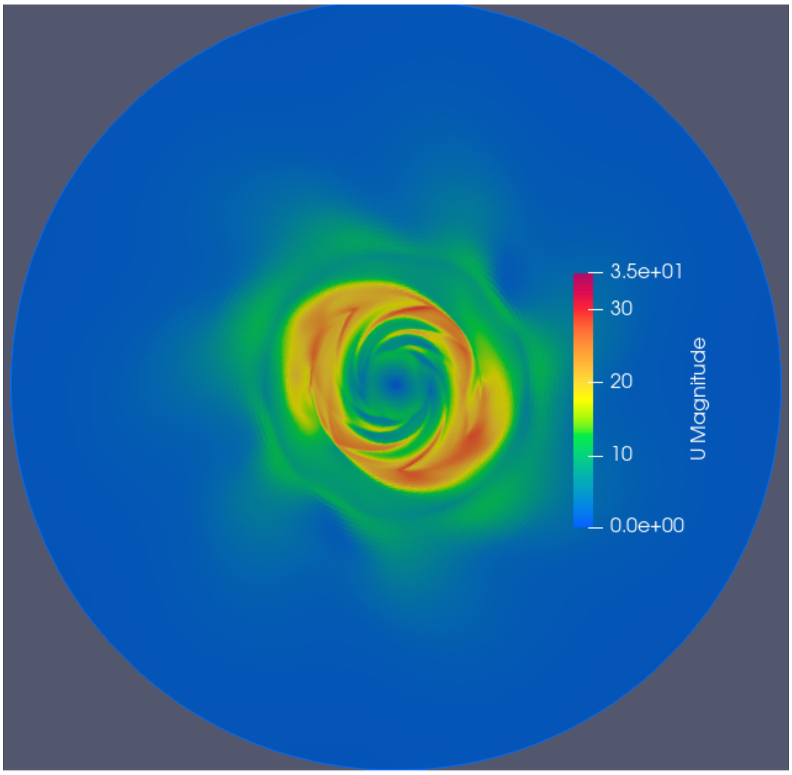


Figure 2.28: First results of CONES optimisation. Slice at $z = 0$, with $\mathbf{D} = (4e7, 2e7, 4e7), D_k = 3e7$ and $D_\omega = -3e7$

We would expected $D_x = D_y$ because the geometry is symmetric, but we see that the optimisation converged to a solution where $D_x \neq D_y$. The local minimum found by the optimisation give us a unsatisfactory solution. D_ω is negative, which is not physical.

2.6 Analysis and Results

Looking at the physics of the pump CFD optimise by DA more in detail, we realize that the IBM method as applied here does not give us a physical result at all and the DA does not allow us to improve this result. It let the flow passing through the external part of the impeller (see Figure 2.29), which is not physical. This imply a recirculation bubble in the diffuser that we would like to avoid.

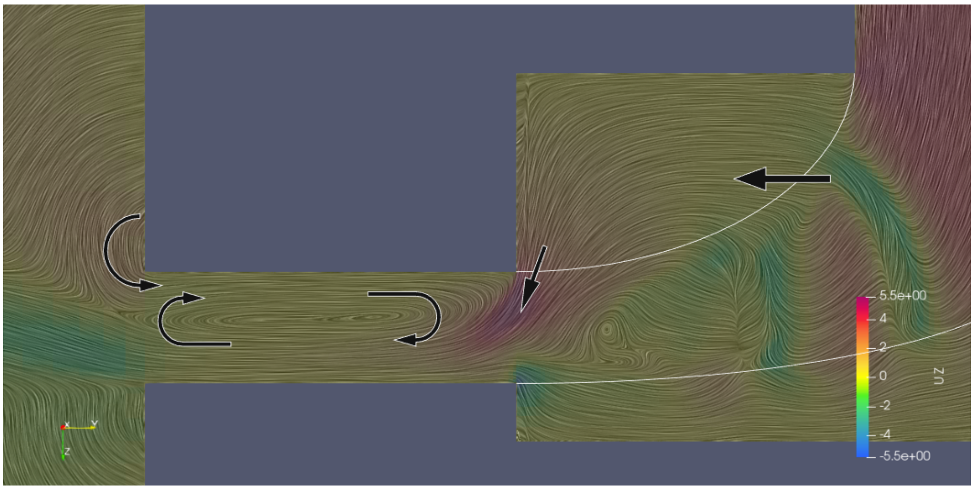


Figure 2.29: Formation of the recirculation bubble in the diffuser

To try to counter this problem, a few strategies are employed, tested separately or in combination (each DA analysis cost a lot of CPU time, so we are limited in the number of analysis we can do):

1. $D_x = D_y$ strategy is to force $D_x = D_y$ (it helps the convergence by the way, by reducing the number of parameters to optimise). In that case, the parameters vector is reduced to $\mathbf{D} = (D_x, D_z, D_k, D_\omega)$.

2. $D_\omega > 0$ strategy is to force $D_\omega > 0$, keeping a variance betweeen each other to allow the parameters to vary.

3. Radial expansion strategy is to use a radial expansion of the five forcing coefficients as illustrated is the Figure 2.30 We used a second order polynomial to represent the radial expansion of the forcing coefficients, for a few raisons:

- it allows to have a more complete representation of the forcing coefficients, which can help to better fit the experimental data;
- it allows to have a smooth variation of the coefficients over the impeller radius, which is more realistic than a constant value;
- it allows to describe pretty weel the geometry in a quadratic way, which is more complete than a sinusoidal representation (see Figure 2.31) which would also require 3 parameters;
- three parameters are not too many so that the optimisation is not too long and the convergence is not too difficult;
- it permit eliminat a step of the value to 0 at the blade tips for a continuity of order 0, which can be important for the numerical stability of the simulation.

In that case, the parameters vector passe from five values to fifteen values.

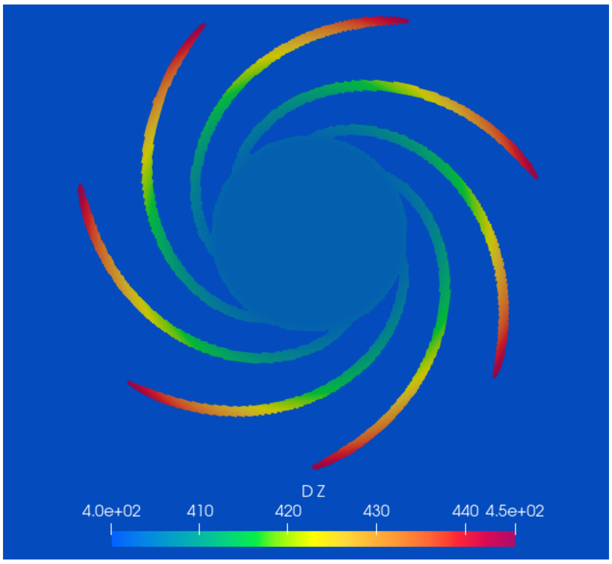


Figure 2.30: Radial expension of Dz over the impeller radius

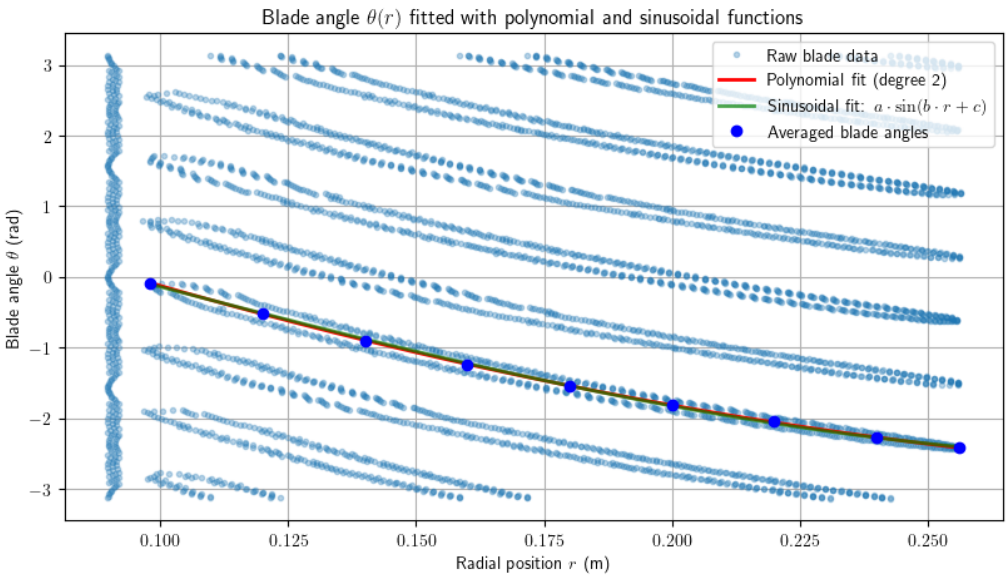


Figure 2.31: Polynomial and sinusoidal interpolation of the blade shape

After applying these strategies, the solution is still not satisfactory, as we can see in the Figure 2.32 where the NRMSEs are still high and the velocity field is still not physical and not converged. The NRMSEs are computed between the numerical simulation and the PIV data, and we can see that they are still high, especially for the radial velocity and velocity over z, because of the no physical recirculation bubble.

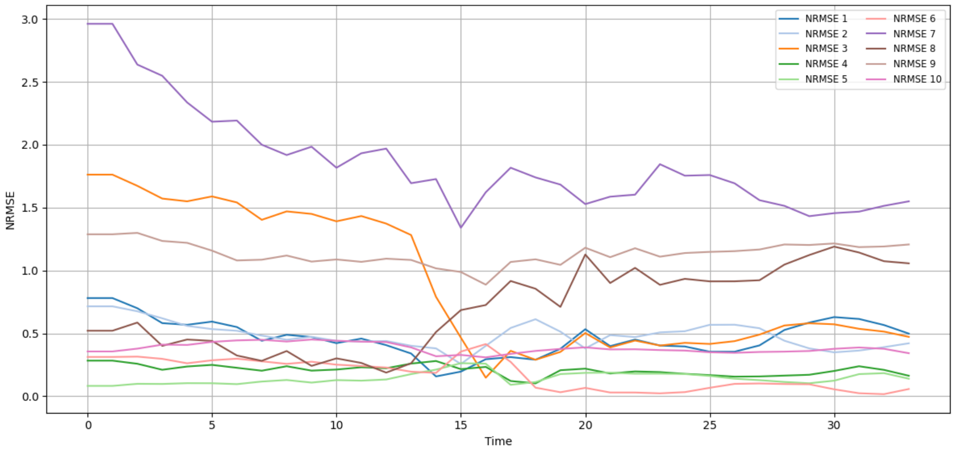


Figure 2.32: Evolution of the NRMSEs over EnKF analysis phases

4. More global interpolation

The folowing explanations are based on the Figure 2.33. The minimum is found based on the blue cercles "coords_seventh" interpolated points and averaged on the "avg_seventh_360" red point localisation. To try to have a more uniform field solution, we can average the parameters over a larger area, i.e., over 360 degrees instead of 360/7 degrees. This allows to have a more global view of the parameters and to avoid local minima (green squares "coords_7" average on the red cercles). But this can also lead to a solution that does not respect the periodicity of the pump, i.e., the solution is not 7 times the same pattern. So we can also average the parameters over 7 segments of 360/7 degrees, which allows to have a more uniform field solution while respecting the periodicity of the pump (orange cruses "coords_360" and purple cercles "avg_7"). In this last case, the length of the coordinate of the observation points is 105 (5 variables, 3 PIV planes, and 7 segments of 360/7 degrees) and the length of the observation vector is 315 (3 dimensions, 105 points).

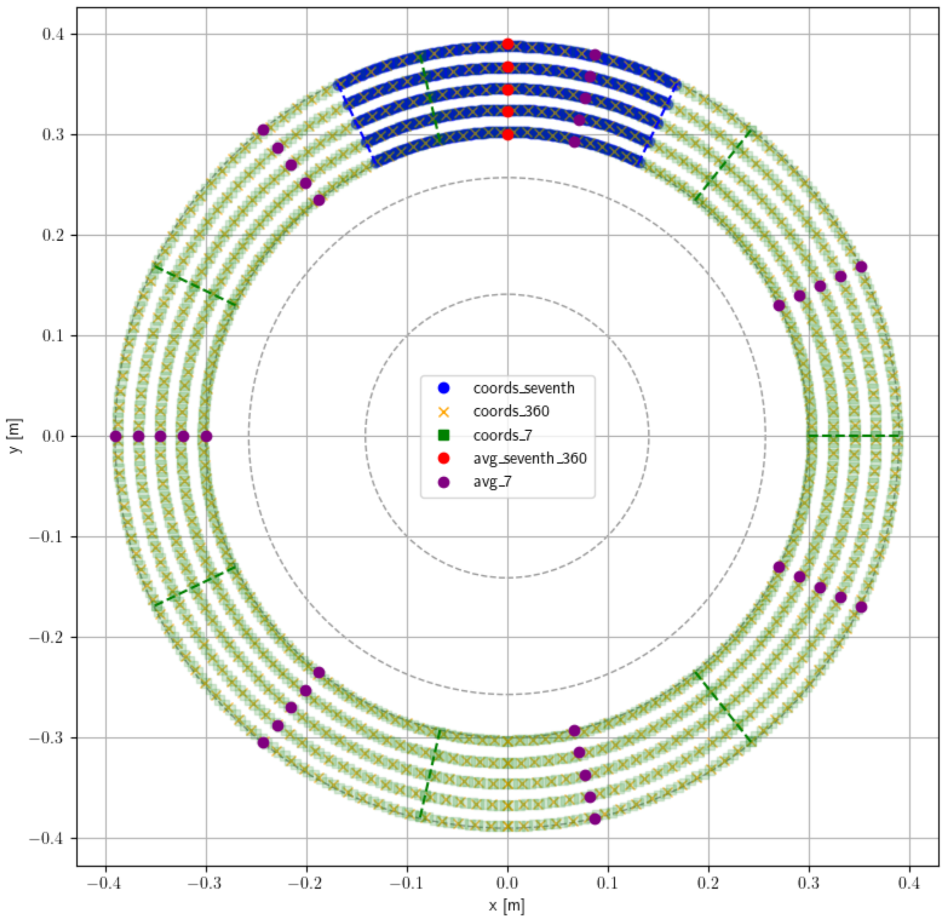


Figure 2.33: Illustration of the augmented coordinates for the interpolation of the field

5.Inflation The inflation strategy (explained id the Equation 2.38) is to use a more complex DA method, such as the EnKF with inflation, which allows to escape from local minima by adding noise to the ensemble members. This can help to explore the parameter space more effectively and find a better solution. But we didn't had this problem in our cas so it hasn't been used.

6.Pressure observation Finnaly, we can also add a pressure observation to the DA, which allows to take into account the pressure difference between the inlet and the outlet of the pump. This can help to improve the convergence of the solution and to have a more physical result. The pressure observation represents the difference of pressure between the inlet of the pipe and the outlet of the diffuser. The numerical coefficient are interpolating, respectively at (0,0,-0.35) and (0,0.390,0). Incertitude on p will be less than the measured one to take more take it account. This implementation add on element oh the observation vector.

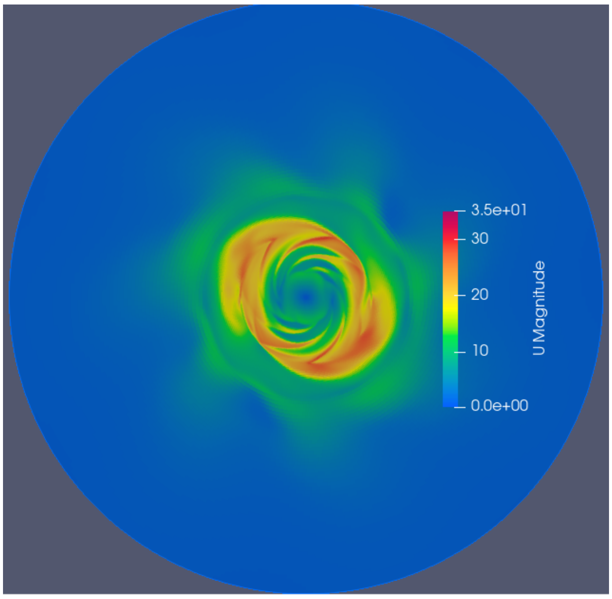


Figure 2.34: Velocity field for Dx = 4e7 and Dy = 2e7

Pb de la DA: plus rapide que DNS mais toujours plus lent que RANS et le temps d'attente et de calcul sur supercalculateur (sur 4000 processeurs) est de l'orde de plusieurs jours. On peut donc se demander si la DA est vraiment utile dans ce cas. En effet, la DA est plus rapide que la DNS mais pas que la RANS. Cependant, il faut prendre en compte le fait que la DA permet de réduire le temps de calcul de la simulation numérique en permettant d'optimiser les paramètres de l'IBM. De plus, la DA peut être utilisée pour d'autres cas d'étude où la DNS est trop coûteuse.

2.6.1 Conclusion and outlook

Conclusion After done a state of art on the topics, numerical ingredients and usefull libraries was presented. An explanation of pre-processing of the experimental data was given, as well as the implementation of the IBM method in OpenFOAM. Then, we presented the first results of the numerical simulation and the first application of the DA method on the centrifugal pump case. The results show that the IBM method as applied here does not give us a physical result and the DA does not allow us to improve this result. The solution is not converged and the velocity field is not physical, with a recirculation bubble in the diffuser. Several strategies were tested to try to improve the solution, but none of them allowed to obtain a satisfactory result. The next step would be to try to improve the IBM method or to try an instationnary (URANS or LES) simulation to see if it can overcome these non-physical results.

Outlook

NOTES PERSO CI-DESSOUS

Pb perso: jusqu'à plusieurs jours d'attente pour lancer une simu.

Reduction of the pipe (with the research of the velocity inlet condition)

Toujours mentionner les paramètres utilisés (dans le controlDict et le CONESDict)

Problème de méthode et pas de miracle ave cette méthode IBM. Interpolation, instationnaire (avantage: on ne contraint plus avec MRF et donc plus facile de réconcilier solution numérique physique et mathématique)

Parler des coefficients de relaxation.

Ouverture: méthode IBM interpolation, article non référencé.

[14] -> ML turbulent channel flow Miguel

"Ouverture / idée": DA temps réel dans avion impossible pour encore longtemps. Solution: Contrôle actif de la puissance en fonction des mesures de pression dans le compresseur: f(obs) -> controle de la puissance Pour ça, se ref au sujet de thèse de Tarane obs pression –> modèle de reconstruction du champ de vitesse –> modèle de reconaissance de patternes de décrochage –> contrôle de la puissance Peut-être faire deux filtres de Kalman imbriqués ? hyperparamètres type fenetre observation, variance initiale, ...

Voir "TD -> Done et "TDSave -> Done" pour voir tt ce que j'ai fais

The random variations are random but with a reproducibility seed (sed) that allows to have the same result each time the simulation is launched.

The theorical average radial speed at the input of the diffuser is equal to

$$\frac{Q_{dif_{in}}}{S_{dif_{in}}} = \frac{Q_{pump_{in}}}{2\pi R_3 h_{dif}} = 3.65 m.s^{-1}$$

With $Q_{pump_{in}} = Q_{dif_{in}}$ because the mass flow rate is constant and the flow is incompressible.// -> No need ??

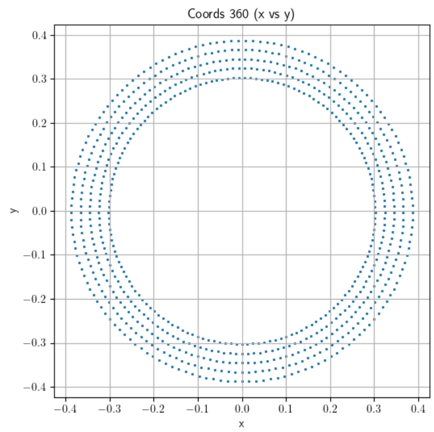


Figure 2.35: Interpolation points over 360 degrees

Methodo (forcage continue (vs interpolation ?)) -> avantages

////Annexe des opérateurs grad, div, nabla, rot, laplacien, laplacien vectoriel, etc...////

The EnKF method will be used to assimilate data from the flow at the observation points and to update the forcing term until it converges to the optimal value. These high-fidelity data are obtained from an experiment explained in the next section 2.3.

Limitations IBM à améliorer avec DA

Graçe aux données xp (mentionné partie 2.1.2, on peut faire de la DA)

Check consistance U, u, P, p, gras, italique,... dans les équations

Comparer les 2-3 résultats: (avant DA, déjà fait section précédente), après simple et "complex" DA

Supposons 1000 itération en moyenne entre chaque phase d'analyse, et 300 phases d'analyse, cela nous fait 300 000 itérations à simuler, soit 93h, sur env. 2800 coeurs, soit 264kh.processeur de calcul.

Conclusion

The goal of this internship was to optimise the numerical performance of a centrifugal pump using DA. After preparing the experimental data and adapted the OpenFoam solver and CONES code, data was able to be assimilate by the EnKF. First results was not convincing, because a recirculation bubble still present in the diffuser. Few more implementation didn't allow the reciculation bubble to break. Then, ...

Technical and human review

Véirifier les consignes

Outlook

Personal reflection

List of Figures

Table des annexes

Annexes

Annex 1 – Batch code

```
#!/bin/bash                      # To be run with bash
#MSUB -r LCP35                   # Job name
#MSUB -o LCP35_%I.out            # Output name. %I is the job ID
#MSUB -e LCP35_%I.err            # Error name. %I is the job ID
#MSUB -q rome                    # Partition
#MSUB -A gen1741                 # Project ID
#MSUB -n 4446                    # Cores nb (35 members x 127 OF) + 1 python
##MSUB -c 2                      # To allocate 2 cores per task (default is 1)
##MSUB -N 1                      # Nb of calcul node (128 cores per node on rome)
#MSUB -T  43200                  # Elapsed time limit in seconds
##MSUB -Q long                   # (upto 3 days)
##MSUB -Q test                   # (short, high priority jobs)
#MSUB -m scratch                 # Excecute in scratch directory
#MSUB -@ name@mail.fr:begin,end # Email address

set -x
cd ${BRIDGE_MSUB_PWD}
module purge
module load gnu/8 mpi/openmpi/4 flavor/python3/cuda python3/3.10.6 openfoam/9
source $FOAM_SOURCE_FILE

./visuNParams.sh                 # Script .sh
ccc_mprun -f app_mySF35.conf     # Run the application with the configuration file
```

Annex 2 – OpenFOAM Directory Tree

```
root
├── 0
│   ├── k
│   ├── omega
│   ├── p
│   └── U
├── 0.orig
│   └── 0
├── system
│   ├── controlDict
│   ├── blockMeshDict
│   ├── decomposeParDict
│   ├── fvSchemes
│   └── fvSolution
├── constant
│   ├── momentumTransport
│   ├── transportProperties
│   └── MRFProperties
├── dummy.foam
├── processor0
├── processor...
└── processorN-1
```

Annex 3 – Extract of conesDict

Extrat form CONES Github[14].

```
paramEstSwitch true; // Switch for parameter estimation
stateEstSwitch false; // Switch for state estimation

// Number of time steps between each analysis phase
observationWindow 800;

// Number of members in the ensemble
ensemble 35;

fineEnsemble 0;
simulationSubDict
{
numberVariablesInState 3;
}

// Number of parameters to optimize with the DA
numberParameters 15;

// Dimensions of the OpenFOAM simulation
nDim 3;

observationSubDict
{
    // Name of the file containing the observation coordinates
    obsCoordFile "obs_coordinates7.txt";

    // Name of the file containing the observation data
    obsDataFile "fieldP7.txt";

    // Number of observation probes for velocity
    numberObsProbesVelocity 105;

    // Specify velocity components in the observation
    obsVelocityComponents (1 1 1);

    // Number of observation probes for pressure
    numberObsProbesPressure 0;

    // Number of observation probes for friction coefficient (default = 1)
    numberOfGlobalObservation 1;

    // pertubation of the obs and diagonal of R matrix for velocity
    velocityObsNoise (10 10 10);

    // pertubation of the obs and diagonal of R matrix for pressure
    pressureObsNoise 0;

    // pertubation of the obs and diagonal of R matrix for the global coefficient
    globalObsNoise 5;

    // Switch to for the obs: true: obs depends on time, false: obs does not depend
       on time
    obsTimeDependency false;

    // The time step of the observations if the case is unsteady
    obsTimeStep 0.004;

    // The start time for the observation data
    obsStartTime 0.0;
```

[14] From:https://gitlab.ensam.eu/pe431/cones-dev/-/blob/pump_clem/tests/pumpIBM/pumpIBMSimple35/system/controlDict

```

    // Def of obs parameter:  U(velocity), p(pressure), Up(velocity + pressure), UCd
        (velocity + drag/friction coeffs), UGlobal(velocity + global obs)
    obsType "UGlobal";

    // Inputs for R are given in absolute values (absVal), percentage (relVal)
    obsNoiseType "absVal";
}


inflationSubDict
{
    // Definition of inflation (0 = stochastic, 1 = deterministic)
    inflationType "deterministic";

    // State inflation (default = 0)
    stateInflation 0;

    // Pre State inflation (considered as model error) (default = 0)
    preStateInflation 0;

    // Do we apply inflation (default = false)
    inflationSwitch false;

    // Parameters inflation
    parametersInflation (0.2 0.2 0.2 0.2 0.2);
}

// Number of txt file written beginning from the last one
numberOutputs 1;

// Print all the debuging messages or not: 1 = printed, 0 = nothing
verbosityLevel 2;
```

Bibliography

[1] Alexis Boudin. "Numerical Methods for Large Eddy Simulation in Turbomachinery (in progress)". PhD thesis.

[2] Philippe Angot, Charles-Henri Bruneau, and Pierre Fabrie. "A Penalization Method to Take into Account Obstacles in Incompressible Viscous Flows". In: *Numerische Mathematik* 81.4 (Feb. 1, 1999), pp. 497–520. ISSN: 0945-3245. DOI: 10.1007/s002110050401. URL: https://doi.org/10.1007/s002110050401.

[3] Mark Asch, Marc Bocquet, and Maëlle Nodet. *Data Assimilation: Methods, Algorithms, and Applications*. Fundamentals of Algorithms 11. Philadelphia: Society for Industrial and Applied Mathematics, 2016. 1 p. ISBN: 978-1-61197-454-6.

[4] A. Dazin et al. "High-Speed Stereoscopic PIV Study of Rotating Instabilities in a Radial Vaneless Diffuser". In: *Experiments in Fluids* 51.1 (July 2011), pp. 83–93. ISSN: 0723-4864, 1432-1114. DOI: 10.1007/s00348-010-1030-x. URL: http://link.springer.com/10.1007/s00348-010-1030-x (visited on 04/18/2025).

[5] Meng Fan et al. "Effect of Leakage on the Performance of the Vaneless Diffuser of a Centrifugal Pump Model". In: (2022).

[6] F. Menter. "Zonal Two Equation K-w Turbulence Models for Aerodynamic Flows". In: *23rd Fluid Dynamics, Plasmadynamics, and Lasers Conference*. DOI: 10.2514/6.1993-2906. eprint: https://arc.aiaa.org/doi/pdf/10.2514/6.1993-2906. URL: https://arc.aiaa.org/doi/abs/10.2514/6.1993-2906.

[7] F. R. Menter. "Two-Equation Eddy-Viscosity Turbulence Models for Engineering Applications". In: *AIAA Journal* 32.8 (1994), pp. 1598–1605. DOI: 10.2514/3.12149. eprint: https://doi.org/10.2514/3.12149. URL: https://doi.org/10.2514/3.12149.

[8] Tom Moussie, Paolo Errante, and Marcello Meldi. "Statistical Inference of Upstream Turbulence Intensity for the Flow Around a Bluff Body with Massive Separation". In: *Flow, Turbulence and Combustion* 113.4 (Nov. 2024), pp. 853–889. ISSN: 1386-6184, 1573-1987. DOI: 10.1007/s10494-024-00573-z. URL: https://link.springer.com/10.1007/s10494-024-00573-z (visited on 05/06/2025).

[9] Charles S Peskin. "Flow Patterns Around Heart Valves: A Numerical Method". In: *Journal of Computational Physics* (1972).

[10] Florence Rabier and Zhiquan Liu. "Variational Data Assimilation: Theory and Overview". In: ().

[11] Ladislao Reti and Francesco Di Giorgio Martini. "Francesco Di Giorgio Martini's Treatise on Engineering and Its Plagiarists". In: *Technology and Culture* 4.3 (Sum. 1963), p. 287. ISSN: 0040165X. DOI: 10.2307/3100858. JSTOR: 3100858. URL: https://www.jstor.org/stable/3100858?origin=crossref (visited on 06/26/2025).

[12] S.R. Shah et al. "CFD for Centrifugal Pumps: A Review of the State-of-the-Art". In: *Procedia Engineering* 51 (2013), pp. 715–720. ISSN: 1877-7058. DOI: 10.1016/j.proeng.2013.01.102. URL: https://www.sciencedirect.com/science/article/pii/S1877705813001033.

[13] M. M. Valero and M. Meldi. "An immersed boundary method using online sequential data assimilation". In: *Journal of Computational Physics* 524 (2025), p. 113697.

[14] Miguel M. Valero and Marcello Meldi. *Enhanced State Estimation for Turbulent Flows Combining Ensemble Data Assimilation and Machine Learning*. Jan. 30, 2025. DOI: 10.48550/arXiv.2501.18262. arXiv: 2501.18262 [physics]. URL: http://arxiv.org/abs/2501.18262 (visited on 07/09/2025). Pre-published.

[15] Roberto Verzicco. "Immersed Boundary Methods: Historical Perspective and Future Outlook". In: *Annual Review of Fluid Mechanics* 55 (Volume 55, 2023 2023), pp. 129–155. ISSN: 1545-4479. DOI: 10.1146/annurev-fluid-120720-022129. URL: https://www.annualreviews.org/content/journals/10.1146/annurev-fluid-120720-022129.

[16] Lucas Villanueva et al. *Augmented State Estimation of Urban Settings Using Intrusive Sequential Data Assimilation*. Jan. 26, 2023. DOI: 10.48550/arXiv.2301.11195. arXiv: 2301.11195 [physics]. URL: http://arxiv.org/abs/2301.11195 (visited on 07/08/2025). Pre-published.

List of acronyms

AI Artificial Intelligence

BC Boundary Conditions

CERFACS "Centre Européen de Recherche et de Formation Avancée en Calcul Scientifique"

CFD Computation Fluid Dynamics

CFL Courant-Friedrichs-Lewy

CHSCTN "Comité d'Hygiène, de Sécurité et des Conditions de Travail National"

CNRS "Centre National de la Recherche Scientifique"

CONES Coupling OpenFoam with Numerical EnvironmentS

CPER "Contrat de Plan État-Région"

CPU Central Processing Unit

CSE "Comité Social et Économique"

CSR Corporate Social Responsibility

CTI "Commission des Titres d'Ingénieur"

DA Data Assimilation

DNS Direct Numerical Simulation

EKF Extended Kalman Filter

EnKF Ensemble Kalman Filter

ENSAM "École Nationale Nupérieure d'Arts et Métiers"

HCERES "Haut Conseil de l'Évaluation de la Recherche et de l'Enseignement Supérieur"

IBM Immersed Boundary Methods

IPSA "Institut Polytechnique des Sciences Avancées"

JFM Journal of Fluid Mechanics

LES Large Eddy Simulation

LMFL "Laboratoire de Mécanique des Fluides de Lille"

Ma Mach number

ML Machine Learning

MPI Message Passing Interface

MRF Multiple Reference Frame

ONERA "Office National d'Études et de Recherche Aérospatiales"

OpenFOAM Open source Field Operation And Manipulation

PDF Probability Density Function

PIV Particle Image Velocimetry

QVT "Qualité de Vie au Travail"

RANS Reynolds-Averaged Navier–Stokes equations

Re Reynolds number

RSE "Responsabilité Sociétale des Entreprises"

RSU "Rapport Social Unique"

SHF "Société Hydrotechnique de France"

SST Shear Stress Transport

TGCC "Très Grand Centre de Calcul du CEA"

U-RANS Unsteady Reynolds-Averaged Navier–Stokes

Résumé Vérifier les consignes

Abstract